

Faculty of Engineering

Department of Informatics Engineering

Software and Information System Engineering



Agentic Retrieval for Semantic Question Answering on Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim

A senior 1 project report - submitted to complete the requirements for obtaining a bachelor's degree in informatics engineering - Software Engineering and Information Systems

Prepared by

Alaa Aldeen Fayez Zaiter

Supervised by

Eng. Anas Abdulaziz

October 2025

SUPERVISION CERTIFICATION

Supervisor Certification

I Certify that the preparation of this project entitled [**Agentic Retrieval for Semantic Question Answering on Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim**]

Prepared by [**Alaa Aldeen Zaiter**] was made under my supervision at Faculty of Informatics Engineering in partial Fulfillment of the Requirements for the Degree of Bachelor of Software and Information system Engineering.

Name:

Signature:

ABSTRACT

Authentic hadith collections form the backbone of Islamic scholarship, with Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim standing as the most reliable sources of Prophetic traditions. These works contain thousands of rigorously authenticated narrations that scholars and students rely on for spiritual guidance, legal reasoning, and historical insight, representing centuries of meticulous scholarly effort in preservation and verification.

Despite their importance, traditional methods for accessing these collections—such as keyword search or manual classification—often fail to capture deeper meanings, contextual nuances, and thematic relationships between narrations. This makes it difficult to answer complex or intent-based questions that go beyond exact wording, highlighting the need for more advanced retrieval approaches.

To address this gap, this project introduces an agentic semantic question-answering system for Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim. The system uses autonomous agents that iteratively search, evaluate, and refine queries using semantic search, keyword matching, and metadata filtering. Through self-assessment and multi-cycle reasoning, it synthesizes information across multiple hadiths and uncovers contextual and thematic links. This approach demonstrates how modern AI can enhance access to Islamic sources while maintaining authenticity and scholarly precision.

ملخص

تُعَدُّ مجموعات الحديث الصحيحة أساس الدراسات الإسلامية، ويأتي صحيح البخاري ومسلم في مقدّمتها باعتبارهما أوثق مصادر السنّة النبوية. تضم هذه المؤلفات آلاف الروايات التي خضعت لتحقيق دقيق، ويعتمد عليها العلماء والطلاب في الاستدلال الشرعي، والتوجيه الروحي، والفهم التاريخي، وهي ثمرة قرون من الجهد العلمي في الجمع والتوثيق.

ورغم أهميتها، فإن الأساليب التقليدية للوصول إلى هذه المصادر، مثل البحث بالكلمات المفتاحية أو التصنيف اليدوي، غالبًا ما تعجز عن استيعاب المعاني العميقة والسياقات الدقيقة والعلاقات الموضوعية بين الأحاديث. ويجعل ذلك الإجابة عن الأسئلة المركّبة أو القائمة على المقاصد أمرًا صعبًا، مما يبرز الحاجة إلى طرق استرجاع أكثر تطورًا.

استجابةً لهذه الحاجة، يقدّم هذا المشروع نظامًا ذكيًا قائمًا على الوكلاء للاستعلام الدلالي في صحيح البخاري ومسلم. يعتمد النظام على وكلاء مستقلين يقومون بالبحث والتقييم وإعادة صياغة الاستعلامات بشكل تكراري باستخدام البحث الدلالي، والمطابقة النصية، وتصفية البيانات الوصفية. ومن خلال التقييم الذاتي والتفكير متعدد المراحل، يتمكّن النظام من ربط الأحاديث ببعضها واستخلاص العلاقات السياقية والموضوعية بينها، مما يوضح كيف يمكن للذكاء الاصطناعي الحديث أن يعزّز الوصول إلى المصادر الإسلامية مع الحفاظ على الأصالة والدقة العلمية.

TABLE OF CONTENT

<i>SUPERVISION CERTIFICATION</i>	<i>I</i>
<i>ABSTRACT</i>	<i>II</i>
<i>ملخص</i>	<i>III</i>
<i>TABLE OF CONTENT</i>	<i>IV</i>
<i>List of tables</i>	<i>VI</i>
<i>List of Figures</i>	<i>VII</i>
<i>Chapter 1 Introduction</i>	<i>1</i>
1. <i>Introduction</i>	<i>2</i>
2. <i>Problem Definition:</i>	<i>3</i>
3. <i>Project objectives:</i>	<i>4</i>
4. <i>Project Scope</i>	<i>5</i>
5. <i>Report organization:</i>	<i>8</i>
6. <i>Summary</i>	<i>8</i>
<i>Chapter 2 Theoretical Background & Literature</i>	<i>9</i>
1. <i>Introduction</i>	<i>10</i>
2. <i>Theoretical Background</i>	<i>10</i>
3. <i>Literature Review</i>	<i>11</i>
4. <i>Research Gap</i>	<i>13</i>
5. <i>Comparative Analysis</i>	<i>14</i>
6. <i>Summary</i>	<i>14</i>
<i>Chapter 3 project management</i>	<i>15</i>
1. <i>Introduction:</i>	<i>16</i>
2. <i>Project management documents:</i>	<i>16</i>
<i>Chapter 4 Data Acquisition and Corpus Processing</i>	<i>24</i>
1. <i>Data Source</i>	<i>25</i>

2. Data Preprocessing and Cleaning.....	25
3. Semantic Chunking Strategy	26
4. Exploratory Data Analysis (EDA).....	30
Chapter 5 System Analysis & Design.....	33
1. System initial analysis.....	34
2. Requirements Modeling	51
3. Test Cases Definition.....	53
4. System Architecture	62
5. State Graph Design (The Core Logic).....	63
6. Multi-Agent Design	65
7. Search & Retrieval Strategy (Hybrid Design).....	66
8. Data Modeling & Schema Design	67
Chapter 6 Implementation.....	74
1. Introduction.....	75
2. Used tools and techniques.....	75
3. Test cases execution	77
4. Requirements Traceability Matrix (RTM).....	83
Chapter 7 Conclusion.....	83
1. Conclusion.....	84
2. Future Work	84
Appendices.....	85
References	86

List of tables

Table 1 Risk Management	21
Table 2 supervisor agent requirements.....	34
Table 3 query analysis agent requirements	36
Table 4 retrieval agent requirements	38
Table 5 evaluation agent requirements.....	40
Table 6 answer synthesis agent requirements.....	41
Table 7 data ingestion component requirements	43
Table 8 infrastructure & framework integration requirements	45
Table 9 user interface requirements.....	45
Table 10 system administration requirements	46
Table 11 non-functional requirements	50
Table 12 test cases definition.....	61
Table 13 Vector Database Document Model.....	69
Table 14 Test cases execution.....	82
Table 15 RTM.....	85

List of Figures

Figure 1 used software development model	22
Figure 2 Gant chart	23
Figure 3 hadith before chunking.....	27
Figure 4 arabic hadith after chunking	28
Figure 5 english hadith after chunking.....	29
Figure 6 chunks diagram	29
Figure 7 distribution of hadith lengths.....	30
Figure 8 total dataset counts.....	31
Figure 9 distribution by book (bukhari).....	31
Figure 10 distribution by book (muslim).....	32
Figure 11 use case diagram.....	51
Figure 12 class diagram.....	52
Figure 13 system architecture	62
Figure 14 state graph design.....	63
Figure 15 multi-agent design.....	65
Figure 16 hybrid search flow	66
Figure 17 strategy pattern	70
Figure 18 single pattern	71
Figure 19 state pattern	72
Figure 20 DAO pattern.....	73

Chapter 1 Introduction

1. Introduction

This chapter presents the Agentic Retrieval System for Semantic Question Answering on Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim, an intelligent digital platform designed to facilitate advanced understanding and exploration of hadith texts through autonomous, iterative question-answering. It highlights the challenges faced by researchers and students when navigating large, complex religious datasets using traditional keyword-based search methods, and introduces the system's main objective: providing accurate, well-referenced, and semantically reasoned answers in Arabic through intelligent agent-driven retrieval workflows.

Unlike conventional retrieval systems that perform single-pass searches, this system employs autonomous intelligent agents that execute multi-cycle retrieval operations. When a user submits a query in Arabic, agents analyze the question, plan retrieval strategies, and iteratively search across both Ṣaḥīḥ collections. After each retrieval cycle, agents self-evaluate the quality and completeness of gathered information, dynamically select between multiple retrieval methods (semantic vector search, keyword matching, metadata filtering), reformulate queries based on discovered context, and autonomously decide when sufficient information has been obtained. This iterative, self-reflective approach enables the system to handle complex queries requiring synthesis across multiple hadiths, thematic connections, and contextual reasoning that traditional single-pass retrieval cannot achieve.

The system leverages advanced natural language processing, vector embeddings, and multi-tool orchestration to analyze user queries, identify semantically relevant hadiths, and generate concise, evidence-based explanations grounded in authentic textual sources. By combining autonomous agent workflows with semantic understanding capabilities, the platform bridges the gap between classical Islamic scholarship and modern artificial intelligence technologies.

Finally, this chapter outlines the overall structure of this report, offering a brief overview of the upcoming chapters that detail the system's requirements, design, implementation, and evaluation.

2. Problem Definition:

Researchers, students, and scholars studying Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim face significant challenges when attempting to locate relevant hadiths and answer complex thematic questions. Traditional search tools rely on keyword matching, which fails to capture semantic meaning and contextual intent. More critically, complex research queries often require multi-step reasoning and iterative exploration that conventional single-pass retrieval systems cannot support. For example, answering "What are the conditions for shortening prayer while traveling?" may require: retrieving hadiths about prayer and travel, identifying specific conditions within results, searching for hadiths elaborating those conditions, comparing across both collections, and verifying completeness—yet traditional systems perform only the first retrieval step, leaving manual investigation to users.

Furthermore, existing digital resources cannot self-assess retrieval quality, reformulate searches based on intermediate findings, or dynamically adjust strategies when results are incomplete or ambiguous. When queries require progressive refinement—where initial results reveal concepts or names that should inform subsequent searches—conventional systems fail to adapt mid-query. This limitation is compounded by challenges in natural Arabic language understanding, where questions are often phrased thematically or contextually rather than using exact hadith terminology.

The fundamental limitation is the inability to reason about retrieval itself—to evaluate whether gathered information is sufficient, identify gaps requiring additional searches, combine multiple retrieval methods strategically, and iteratively refine queries until complete answers emerge. Simple semantic search improvements cannot address these challenges because they operate on a single-pass paradigm without mechanisms for self-correction or progressive information gathering.

Therefore, there is a clear need for an agentic retrieval and reasoning system that employs autonomous agents capable of: executing multi-cycle retrieval with self-evaluation, dynamically selecting and combining retrieval strategies, reformulating queries based on intermediate results, and autonomously determining when sufficient information has been gathered to generate accurate, well-referenced answers.

3. Project objectives:

The primary objective of this project is to develop an intelligent Agentic Retrieval System for Semantic Question Answering on Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim. The system aims to bridge the gap between traditional Islamic scholarship and modern artificial intelligence by enabling Arabic-speaking users to query authentic hadith texts through autonomous, iterative retrieval workflows powered by externally-hosted, serverless AI endpoints.

To achieve this vision, the system will:

- **Implement a Multi-Agent Architecture:** Utilize five specialized agents (Supervisor, Query Analysis, Retrieval, Evaluation, Synthesis) to manage a stateful, iterative workflow.

- **Handle Complex Queries:** Go beyond simple search to support thematic, comparative, and narrator-based queries by decomposing them into multiple steps.
- **Enforce Zero-Hallucination:** Generate answers that are strictly grounded in the retrieved Hadith texts, with a policy of never fabricating information.
- **Provide Full Citations:** Produce complete, accurate citations for all referenced Hadiths.
- **Deploy a Scalable AI Infrastructure:** Leverage an external, serverless GPU provider to host and serve the AI models, ensuring scalability and cost-efficiency.

4. Project Scope

4.1 Phase 1 Scope

The initial phase establishes an autonomous multi-agent retrieval system for Arabic hadith question-answering over Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim. The system employs specialized intelligent agents orchestrated through LangGraph stateful workflows, where each agent autonomously reasons, selects tools, and collaborates to answer complex queries through iterative retrieval cycles. Unlike traditional single-pass RAG systems, this platform ensures strict grounding in source texts with zero-hallucination policies, utilizing externally-hosted, serverless AI endpoint. The system will:

Core Multi-Agent Architecture:

- Implement a LangGraph-based multi-agent system with five specialized agents:
 - **Supervisor Agent:** Orchestrates workflow execution, routes tasks between agents, manages iteration limits, and enforces stopping conditions

- **Query Analysis Agent:** Processes user input, detects typos, normalizes text, classifies intent, decomposes multi-part questions, and identifies input sources
- **Retrieval Agent:** Plans and executes retrieval strategies, dynamically selects tools (semantic search, keyword search, metadata filtering, hybrid search), and gathers results across collections
- **Evaluation Agent:** Assesses retrieval quality, identifies information gaps, computes confidence scores, validates grounding, and determines iteration necessity
- **Synthesis Agent:** Combines information from multiple hadiths, generates strictly grounded Arabic answers, produces complete citations, and enforces no-hallucination policies
- **Knowledge Base:** Ingest and index the complete Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim collections.
- **Retrieval Capabilities:** Implement a multi-tool retrieval agent that can autonomously choose between semantic search, keyword search, and metadata filtering.
- **Serverless Technical Infrastructure:** Deploy the system as a decoupled architecture:
 - A lightweight FastAPI application for orchestration.
 - A serverless Modal.com service to host the Qwen2.5 LLM and the embedding model on-demand.
- **Observability:** Integrate the system with LangSmith to trace and debug all agent decisions and workflows.

4.2 Future Expansion (Phase 2 and beyond)

The system architecture is designed to support:

- Extension to **complete Kutub al-Sittah** (six canonical hadith collections including Sunan Abu Dawood, Sunan al-Tirmidhi, Sunan al-Nasa'i, Sunan Ibn Majah)
- **Multilingual question-answering** supporting English, French, Urdu, and other languages with cross-lingual retrieval
- **Advanced contradiction resolution** integrating jurisprudential context and scholarly interpretations
- **Hadith authenticity grading** with display of authentication chains and scholarly assessments
- **User management system** with authentication, personalized search history, bookmarking, and annotation capabilities
- **Enhanced UI/UX** including voice interaction, mobile applications, and interactive visualizations
- **Scholarly commentary integration** linking hadiths to classical and contemporary Islamic scholarship
- **Export capabilities** (PDF reports, academic citation formats, API integrations for third-party applications)
- **Analytics dashboards** for administrators showing query patterns, system performance, and usage statistics
- **Fine-tuning pipelines** for domain-specific model optimization on hadith corpora
- **Collaborative research tools** enabling shared annotations, discussions, and collective knowledge building
- **Real-time notifications** for new relevant hadiths or scholarly updates
- **Advanced semantic features** including conceptual clustering, thematic mapping, and knowledge graph construction

5. Report organization:

The report will be organized in the following chapters:

Chapter 1: Introduction

Chapter 2: Theoretical Background and Literature Review

Chapter 3: Project Management

Chapter 4: Data Acquisition and Corpus Processing

Chapter 5: System Analysis & Design

Chapter 6: Practical Implementation

Chapter 7: Conclusion

6. Summary

This chapter introduced the Agentic Retrieval System for Semantic Question Answering on Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim, explained the motivation and problem definition, and defined the project's objectives and scope. It also outlined the overall structure of this report. The next chapter presents the fundamental concepts and related literature that form the theoretical foundation of the proposed system.

Chapter 2 Theoretical Background & Literature

1. Introduction

This chapter establishes the scientific and technical foundations of the **Agentic Retrieval System for Semantic Question Answering**. It is divided into two main sections:

1. **Theoretical Background:** Explains the core technologies used in the system, such as Vector Embeddings, Semantic Search, and Agentic Workflows.
2. **Literature Review:** Critically analyzes previous academic studies and existing systems in the field of Arabic Information Retrieval (AIR) and Islamic Question Answering (IQA), highlighting their limitations and identifying the **Research Gap** that this project aims to fill.

2. Theoretical Background

This section provides an overview of the fundamental concepts that drive the proposed system.

2.1 Semantic Search and Vector Embeddings

Traditional information retrieval relies on **Lexical Search** (e.g., BM25), which matches exact keywords. However, this approach fails to capture the rich morphology of the Arabic language (synonyms, root variations). **Semantic Search** addresses this by using **Vector Embeddings**. Large Language Models (LLMs) like **BERT** and its Arabic variants (AraBERT, Marbert) map text into high-dimensional vector spaces where semantically similar texts are closer together, even if they share no common words. This allows the system to understand that "Wudu" (الوضوء) and "Ablution" (الطهارة) are related concepts.

2.2 From RAG to Agentic RAG

Retrieval-Augmented Generation (RAG) improves LLM reliability by grounding answers in external data. However, standard RAG processes are linear (Retrieve → Generate) and "blind" to their own mistakes. **Agentic RAG** introduces a paradigm shift. It employs **Autonomous Agents** capable of:

- **Reasoning (ReAct):** Breaking down complex questions into sub-tasks.
- **Self-Reflection:** Evaluating retrieved documents and deciding if they are sufficient.
- **Tool Use:** Dynamically selecting between Keyword Search, Semantic Search, or Metadata Filters based on the query's nature. This system utilizes **LangGraph** to orchestrate these agents in a cyclic, stateful workflow, allowing for iterative refinement of results.

3. Literature Review

This section reviews recent academic contributions to Arabic and Islamic text processing, categorized into three key areas.

3.1 Semantic Search in Islamic Texts

Early attempts at digitizing Islamic texts focused on keyword indexing. **Sudani (2020)** proposed a generic semantic retrieval system for Arabic based on context, but it lacked deep reasoning capabilities. More recently, researchers have applied Deep Learning to Hadith. **Torki et al. (2023)** introduced the *Author-Based Hadith Classification Dataset (ABCD)*, demonstrating that Deep Learning models (DL) outperform traditional Machine Learning (ML) in analyzing the *Sanad* (chain of narrators) with an F1-score of 92%, while ML performed better on the *Matn* (content). Similarly, studies using **ArabicBERT** for Hadith authenticity classification achieved 86% accuracy. However, these studies focused primarily on

Classification (Sahih vs. Daif) rather than **Question Answering** or extracting rulings from the texts.

3.2 Challenges in Arabic Question Answering (QA)

Developing QA systems for Arabic, particularly for religious texts, faces significant hurdles. A survey by **Alnefaie et al. (2023)** highlighted the scarcity of reliable **Hadith QA Corpora**, noting that most existing datasets are small or limited to specific chapters (like *Al-Baqarah* in the Quran). Furthermore, a critical risk in using Generative AI for religion is **Hallucination**. A recent analysis (2025) categorized LLM errors in religious contexts into "Fabricated Hadith," "Incorrect Attribution," and "Distortion of Meaning," emphasizing the need for systems that enforce **Strict Grounding** rather than relying on the model's parametric memory.

3.3 Agentic RAG vs. Enhanced RAG

The distinction between "Enhanced RAG" (using fixed modules for re-ranking) and "Agentic RAG" (autonomous decision-making) is a cutting-edge research topic. A 2025 systematic review indicated that while **Agentic RAG** incurs higher latency and computational cost, it significantly outperforms static systems in handling **Ambiguous Intents** and **Complex Reasoning** tasks. This finding supports our project's architectural choice: focusing on accuracy and reasoning depth (via Agents) is prioritized over speed for the sensitive domain of Fatwa and Hadith research.

4. Research Gap

Based on the review of previous studies, the following gaps were identified:

1. **Lack of Reasoning:** Most existing Hadith systems (e.g., Sunnah.com) are search engines, not answering engines. They return documents, leaving the user to synthesize the answer.
2. **Static Retrieval:** Current academic approaches use single-pass retrieval. If the initial search fails, the system returns no results rather than reformulating the query (Self-Correction).
3. **Authentication Granularity:** Few systems combine *Semantic Search* (for meaning) with *Metadata Filtering* (for narrator chains) in a unified workflow.

Contribution: This project addresses these gaps by building an **Agentic System** that not only retrieves Hadiths but iteratively reasons, evaluates relevance, and synthesizes grounded answers with zero hallucination.

5. Comparative Analysis

The following table positions the proposed system against existing solutions:

Feature	Sunnah.com	HadithEnc.com	ChatGPT	Proposed System
Search Mechanism	Keyword Match	Keyword + Categorical	Parametric Knowledge	Hybrid (Semantic + Keyword)
Reasoning Capability	None	None	General Reasoning	Agentic (Iterative & Reflective)
Hallucination Control	N/A (Static DB)	N/A (Static DB)	High Risk	Strict Grounding (Zero-Hallucination)
Arabic Support	Excellent	Excellent	Good	Specialized (Pre-processing)
Citation Style	Direct Link	Direct Link	Generic	Academic & Metadata-Rich
Self-Correction	No	No	No	Yes (Evaluation Agent)

6. Summary

This chapter provided the theoretical framework for the project, covering vector embeddings and agentic workflows. It critically reviewed the literature, showing that while significant progress has been made in Arabic classification and static search, there is a clear absence of autonomous, reasoning-based QA systems for Hadith. The proposed system aims to bridge this gap by leveraging the latest advancements in Agentic AI.

Chapter 3 project management

1. Introduction:

In this chapter, we will explore the project management phase, a crucial element in ensuring the project's success. We will focus on scope determination, initial analysis, and system structuring, laying the foundation for a well-organized and efficient development process.

2. Project management documents:

2.1 Project charter:

A project charter is a formal document that serves as an official authorization for the start of a project. It acts as a reference point throughout the project, providing a clear understanding of the project's purpose and establishing a foundation for decision-making and project governance.

<i>Project title</i>	Agentic Retrieval for Semantic Question Answering on Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim
<i>Project start date</i>	October – 18 – 2025
<i>Project finish date</i>	January – 18 – 2026
<i>Project manager</i>	Eng. Anas Abdulaziz
<i>Project objectives</i>	The primary objective of this project is to develop an intelligent Agentic Retrieval System for Semantic Question Answering on Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim. The system aims to bridge the gap between traditional Islamic scholarship and modern artificial intelligence by enabling Arabic-speaking users to query authentic hadith texts semantically rather than through basic keyword search.
<i>Approach</i>	1. Define the project scope, objectives, and data sources.

2. Analyze and document system requirements, emphasizing Arabic semantic understanding and reasoning.
3. Design the architecture integrating **LlamaIndex**, **LangChain** and **LangGraph**, and a **vector database** for RAG-based retrieval.
4. Develop the first increment focusing on data preprocessing, chunking, and Arabic embeddings creation.
5. Test and validate retrieval accuracy for basic queries.
6. Develop the second increment focusing on agentic retrieval, reasoning, and contradiction analysis.
7. Perform testing on semantic accuracy, reliability, and reference validation.
8. Document all phases, results, and evaluation outcomes comprehensively.

Roles and responsibilities:

Name	Role	Responsibility
Eng. Anas Abdulaziz	Supervisor	Highly Project management and work monitoring.
Alaa Zaiter	Engineer	Backend development – documentation – frontend development – deployment.

2.2 The SOW document:

Statement of Work is a comprehensive document that defines the scope of work for a project. It outlines the specific tasks, deliverables, timeline, and responsibilities. The SOW document provides a clear understanding of what needs to be accomplished, the project's objectives, and the criteria for success.

Project Title: Agentic Retrieval for Semantic Question Answering on Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim

Description and Objectives:

The primary objective of this project is to develop an intelligent **Agentic Retrieval System for Semantic Question Answering on Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim**. The system aims to bridge the gap between traditional Islamic scholarship and modern artificial intelligence by enabling Arabic-speaking users to query authentic hadith texts through autonomous, iterative retrieval workflows rather than through basic keyword search or single-pass semantic retrieval.

Project Scope:

- Implement a LangGraph-based multi-agent system with five specialized agents: **Supervisor Agent, Query Analysis Agent, Retrieval Agent, Evaluation Agent, Synthesis Agent**.
- Deploy **stateful cyclic workflows** where agents communicate through shared state, execute iterative refinement cycles, and make autonomous decisions about retrieval strategies and stopping conditions
- Integrate **LangSmith for observability** to trace agent decisions, monitor execution paths, track token usage, and enable debugging of complex multi-agent interactions

<i>Project goals:</i>	• Build an AI system that enables semantic exploration of authenticated Arabic hadith collections. • Demonstrate the use of agentic retrieval for reasoning across multiple sources. • Ensure accuracy and reliability of hadith references, including metadata and source verification. • Provide a scalable system architecture that can be extended in future phases to include additional collections or advanced reasoning tasks.
<i>Project Deliverables:</i>	○ <i>Project plan.</i> ○ <i>Working software.</i> ○ <i>Final project report.</i>
<i>Technology and Tools:</i>	Programming Languages: Python, HTML, CSS, JavaScript. Frameworks & Libraries: LlamaIndex, LangChain, FastAPI, Chroma (vector DB), Streamlit or Gradio. AI Models: Qwen2.5-14b and BGE-m3. Database: JSON datasets for hadiths, Chroma vector database for embeddings. Development Environment: VS Code, Google Colab for experimentation
<i>Assumptions:</i>	■ The project is developed solo by the student with 32 hours/week availability. ■ Arabic datasets for <i>Ṣaḥīḥ al-Bukhārī</i> and <i>Ṣaḥīḥ Muslim</i> are complete and accessible in JSON format. ■ Continuous access to AI services for embeddings and language models. ■ The supervisor will provide regular feedback and guidance throughout the project phases. ■ Computing resources are sufficient for embedding and retrieval tasks.

<i>Project Resources</i>	<i>Human Resources:</i> <i>Eng. Anas Abdulaziz – Project Manager & supervisor.</i> <i>Alaa Zaiter – SE Developer</i>
<i>Schedules:</i>	<i>Project Start Date: October 18, 2025</i> <i>First project Seminar: November 11, 2025</i> <i>Second project Seminar: November 27, 2025</i> <i>Project Technical Interview: January 10, 2026.</i> <i>Project Finish Date: January 18, 2026.</i>

2.3 Risk Management:

the process of identifying, evaluating, and mitigating potential risks that could impact the success of the project or the team.

Risk_ID	Rk-01	Rk-02	Rk-03
Risk title	Small Team Size (Only One Member).	Unclear Project Scope	Learning and Adopting New Technologies
Risk Description	Since the project is developed by a single member, any delay, illness, or workload issue may directly affect development progress, documentation quality, and overall project completion timeline.	A misunderstanding of the project scope at the beginning could lead to major errors in development.	The project requires developing advanced AI software, but my initial knowledge in this area is limited.

Raised Date	18/10/2025	28/10/2025	25/10/2025
Tracking Frequency	weekly	daily	weekly
State	active	closed	Under Mitigation
Impact	high	medium	high
Mitigation Plan	Maintain a detailed weekly schedule with prioritized milestones, perform continuous progress tracking, and seek early feedback from the supervisor to avoid delays.	Conduct regular meeting with project supervisor at the end of each phase to ensure alignment	Engage in continuous learning and self-development.

Table 1 Risk Management

3. Project development life cycle model:

3.1 Determine development model based on system requirements and structure:

After conducting a high-level analysis, eliciting system requirements, and decomposing the system into its main components, the next step is to determine the most suitable software development process for our system. Given the nature of our system and its requirements, we have chosen the Incremental Development Model as the most appropriate approach.

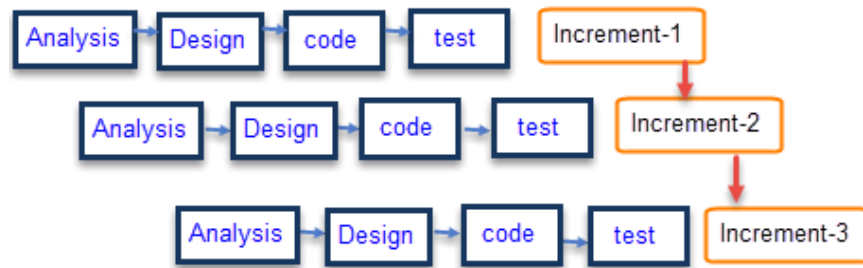


Figure 1 used software development model

The incremental approach aligns well with our system's structure since it consists of separate, independent components that can be developed individually as standalone sub-projects. This model allows us to gradually build and integrate system components, ensuring flexibility and iterative improvements.

3.2 Incremental project management:

Unlike traditional models where the entire project is planned upfront, incremental project management plans and executes in phases where the focus is on achieving key business objectives and delivering benefits to the customer in a series of small increments or stages. The Project Charter and Statement of Work (SOW), created at the project's beginning, may evolve to accommodate updates and refinements based on system needs and development progress.

3.3 Initial Project Plan – Gantt chart:

A document outlining tasks, deadlines, and resources needed to achieve project objectives, serving as a roadmap for successful project execution, and reflecting the system development model.

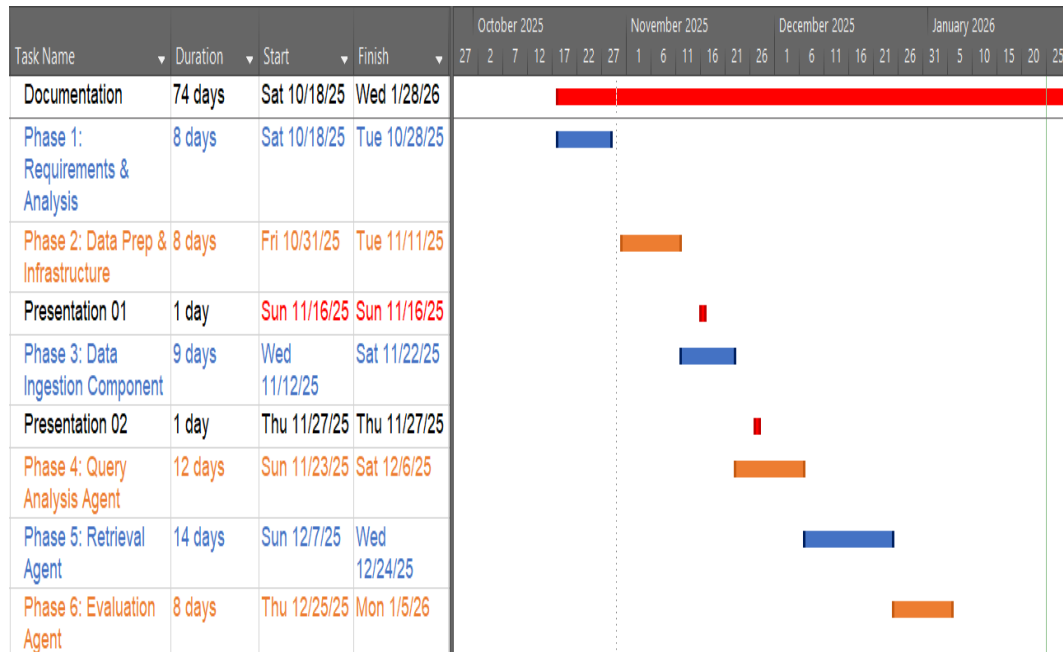


Figure 2 Gantt chart

Chapter 4 Data Acquisition and Corpus Processing

1. Data Source

The core knowledge base of the system is constructed from the two most authentic collections in Islamic tradition: **Sahih al-Bukhari** and **Sahih Muslim**. These collections were acquired in a structured JSON format to facilitate programmatic access and manipulation.

- **Sahih al-Bukhari**: Contains approximately 7,277 hadiths.
- **Sahih Muslim**: Contains approximately 7,459 hadiths.

The data was ingested in a machine-readable format containing critical metadata fields such as the Hadith ID, Book ID, Chapter ID, Narrator chain, and the full text in both Arabic and English. This structured approach ensures that every retrieved answer can be traced back to its specific source references.

(**Note:** A detailed list of the datasets, including download links and citations for the raw data repositories, is provided in **Chapter 6: References**.)

2. Data Preprocessing and Cleaning

Raw Arabic text presents unique challenges for Information Retrieval (IR) due to its rich morphology, diacritics (Tashkeel), and orthographic variations. To ensure high recall in search operations, a specialized preprocessing pipeline was implemented using the `arabic_processing.py` module.

The cleaning process involved the following stages:

1. **Diacritic Removal with Semantic Preservation:** Standard diacritic removal often strips the Shadda (ّ), which can alter the meaning of words (e.g., differentiating between "taught" *allama* and "knew" *alima*). Our custom `remove_diacritics_keep_shadda` function removes decorative diacritics (Fatha, Kasra, Damma) while preserving the Shadda to maintain semantic integrity.

2. Orthographic Normalization: To unify inconsistent writing styles, we applied "Deep Normalization" for search indexing:
 - a. Alef Normalization: Variants like (أ, إ, إ) were normalized to a bare Alef (ا).
 - b. Ta-Marbuta Normalization: The character (ة) was normalized to (ت) to match common user typing patterns.
 - c. Hamza Standardization: Varied forms of Hamza (أ, إ, إ) were mapped to their base letters.
3. Noise Reduction: Invisible Unicode directional marks (LTR/RTL markers) were stripped using the `remove_directional_marks` function to prevent encoding errors during vector embedding.

3. Semantic Chunking Strategy

One of the critical challenges in RAG systems is handling variable-length texts. Hadiths can range from a single sentence to several pages (e.g., the Hadith of Intercession or the Treaty of Hudaibiyyah). Passing entire long texts to the embedding model can exceed the context window or dilute the semantic density of the vector.

To address this, we implemented a Semantic Chunking Strategy rather than naive character-based splitting.

1. Sentence Boundary Detection: Instead of arbitrarily cutting text at fixed character limits (which risks breaking sentences mid-meaning), the system uses regex-based boundary detection (`ARABIC_SENTENCE_BOUNDARIES`) to identify logical stops. It respects Arabic punctuation marks such as the Arabic comma (،), full stop (.), and question mark (؟).
2. Overlap for Context Continuity: We employed a sliding window approach with an overlap of approximately 100 characters (or 1 sentence). This

ensures that if a semantic concept spans across the cut point of two chunks, the context is preserved in both, preventing information loss.

3. Metadata Preservation: Crucially, every chunk retains the full metadata of its parent Hadith (Book ID, Chapter Title, Narrator). A `parent_hadith_id` field links all chunks back to the original source, ensuring that even if a specific chunk is retrieved, the system can cite the full Hadith correctly.

- Max Chunk Size: ~800 characters (approx. 400 tokens) to fit optimally within the bge-m3 embedding model's context window.

3.1 Data Transformation Example

To illustrate the transformation pipeline, we present a sample instance of a Hadith before and after processing. This demonstrates how the raw nested structure is flattened and enriched with metadata to support efficient retrieval.

- A. **Raw Input (Before Processing)** The following JSON represents a raw entry from the Sahih al-Bukhari dataset. Note the nested structure of the English translation and the lack of explicit flattened metadata.

Example Input (Hadith ID: 15):



```

1  {
2    "id": 15,
3    "idInBook": 15,
4    "chapterId": 2,
5    "bookId": 1,
6    "arabic": "حَدَّثَنَا يَعْقُوبُ بْنُ إِبْرَاهِيمَ... لَا يُؤْمِنُ أَحَدُكُمْ حَتَّى أَكُونَ",
7    "english": {
8      "narrator": "Narrated Anas:",
9      "text": "The Prophet (ﷺ) said \"None of you will have faith till he loves me more than his father, his children and all mankind.\""}
10  }
11 }
```

Figure 3 hadith before chunking

B. Processed Output (After Chunking & Enrichment) The system processes this input to produce search-ready "Documents". Key transformations include:

1. Metadata Enrichment: Fields like `chapter_title_en` ("Belief") and `chapter_title_ar` ("كتاب الإيمان") are injected to aid filtering.
2. Structure Flattening: The nested English object is separated into its own document.
3. Parent Linking: A `parent_hadith_id` (`sahih_al-bukhari_1_15`) is assigned to link both the Arabic and English chunks back to the original source.

Example Output (Arabic Chunk):



```

1  {
2    "chunk_id": "sahih_al-bukhari_1_15_arabic_chunk_0",
3    "text": "حَدَّثَنَا يَعْقُوبُ بْنُ إِبْرَاهِيمَ... لَا يُؤْمِنُ أَحَدُكُمْ حَتَّى أَكُونَ",
4    "language": "arabic",
5    "chunk_index": 0,
6    "total_chunks": 1,
7    "is_chunked": false,
8    "collection": "Sahih al-Bukhari",
9    "chapter_title_en": "Belief",
10   "parent_hadith_id": "sahih_al-bukhari_1_15"
11 }

```

Figure 4 arabic hadith after chunking

Example Output (English Chunk):

```
1  {
2    "chunk_id": "sahih_al-bukhari_1_15_english_chunk_0",
3    "text": "The Prophet (ﷺ) said \"None of you will have faith
4    till he loves me more than his father, his children and all mankind.\",
5    "language": "english",
6    "chunk_index": 0,
7    "total_chunks": 1,
8    "is_chunked": false,
9    "collection": "Sahih al-Bukhari",
10   "chapter_title_en": "Belief",
11   "parent_hadith_id": "sahih_al-bukhari_1_15"
12 }
```

Figure 5 english hadith after chunking

Chunked vs. Unchunked:

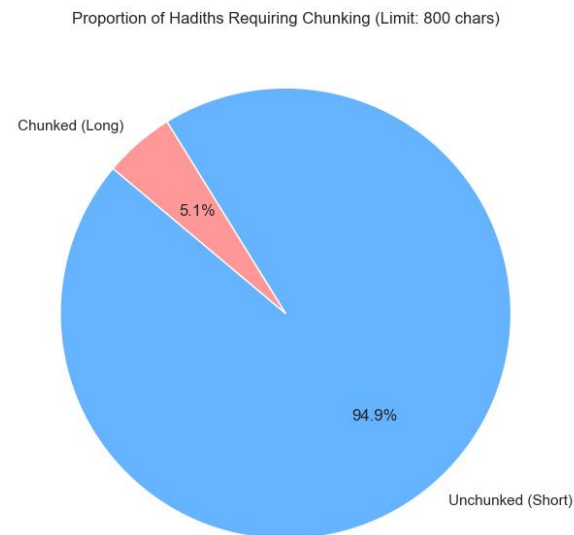


Figure 6 chunks diagram

4. Exploratory Data Analysis (EDA)

To understand the characteristics of the corpus and optimize the retrieval parameters, we performed an Exploratory Data Analysis (EDA).

Chart 1: Distribution of Hadith Lengths

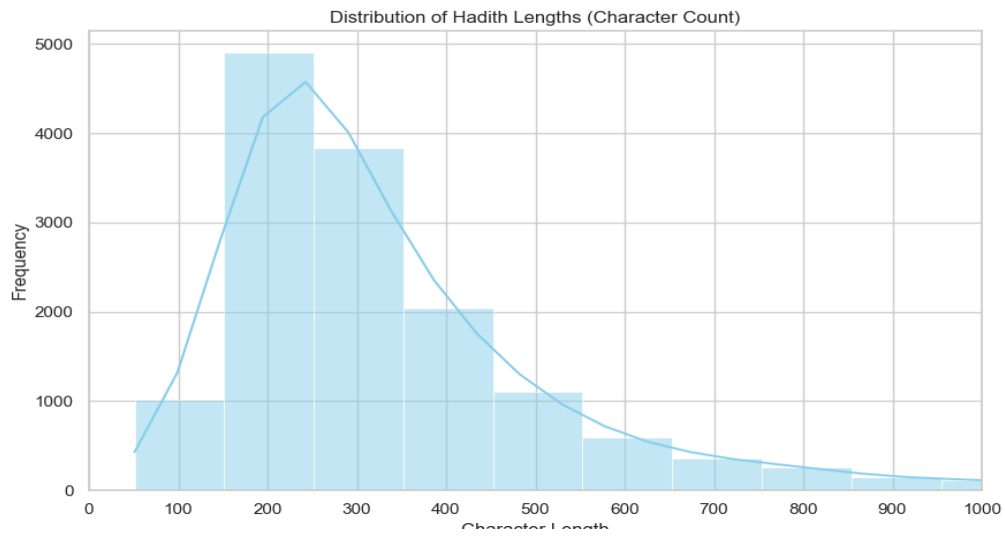


Figure 7 distribution of hadith lengths

Analysis: The histogram reveals a long-tail distribution. The majority of Hadiths are concise (under 500 characters), representing short sayings or rulings. However, the tail extends significantly to include "Long Hadiths" (e.g., >3,000 characters). This validates the necessity of our Chunking Strategy; without it, these semantically rich long narratives would be truncated by the embedding model.

Chart 2: Total Dataset Counts

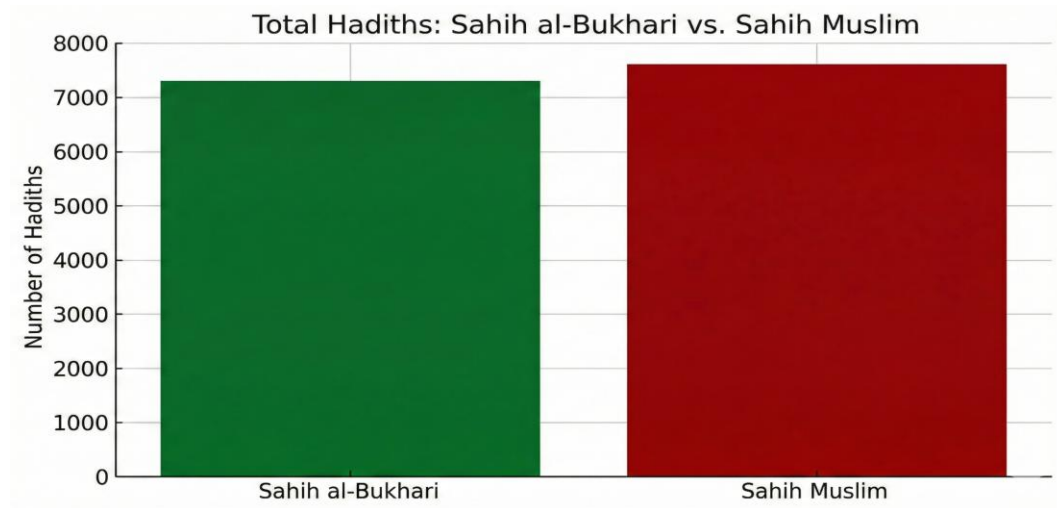


Figure 8 total dataset counts

Analysis: The dataset comprises approximately 7,277 narrations from Sahih al-Bukhari and a comparable set from Sahih Muslim. This volume provides a comprehensive coverage of Islamic jurisprudence (Fiqh) and theology (Aqeedah), sufficient for a robust RAG system.

Chart 3 & 4: Distribution by Book (Kitab)

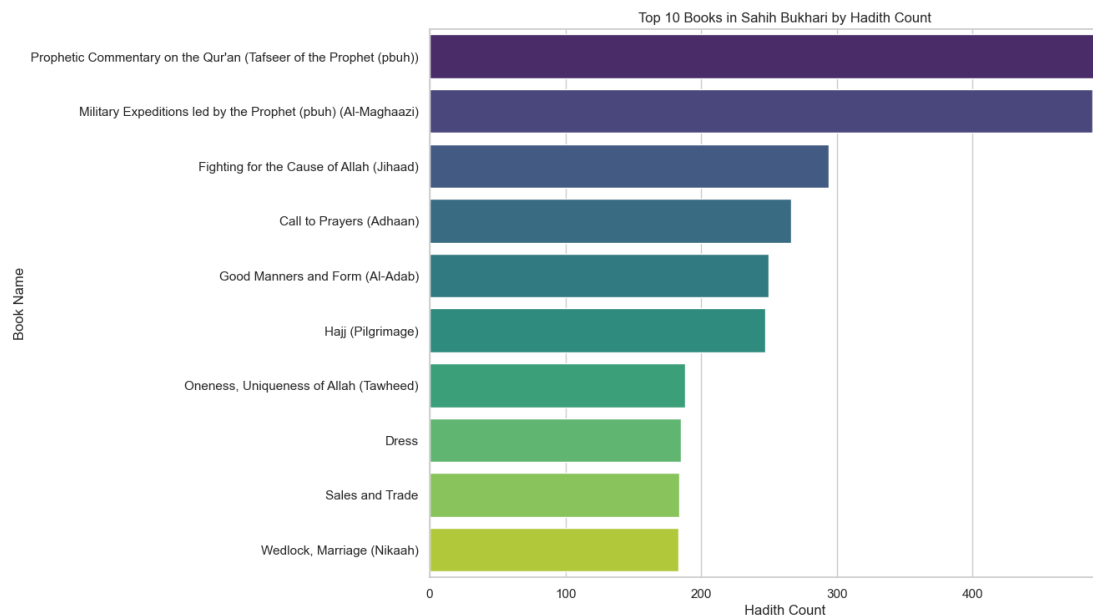


Figure 9 distribution by book (bukhari)

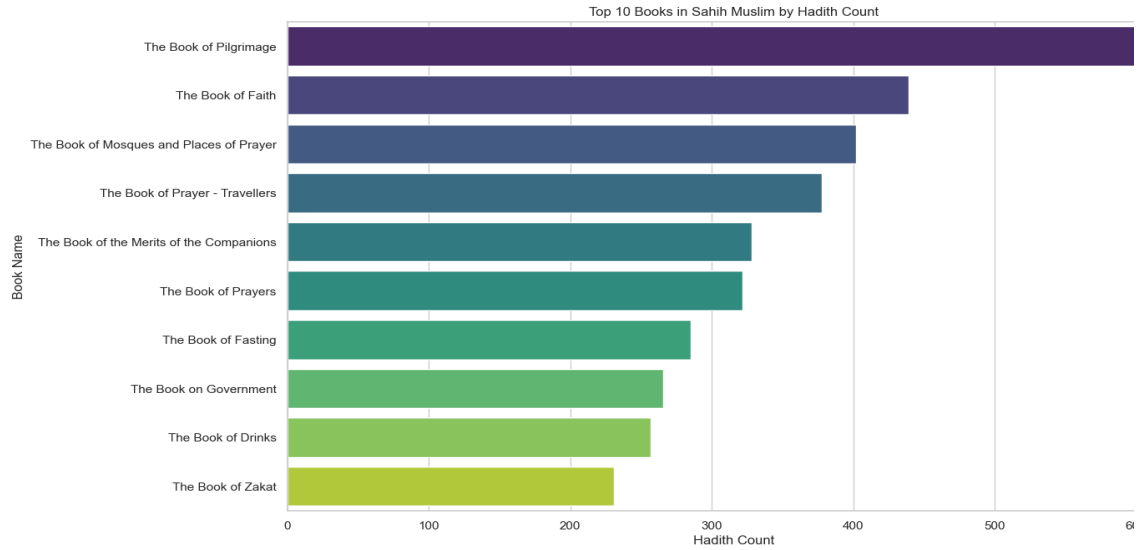


Figure 10 distribution by book (muslim)

Analysis: Analyzing the distribution by "Book" (Kitab) shows that topics related to Tafseer of the prophet , Al-Maghaazi, Pilgrimage ,and Faith contain the highest volume of narrations. This insight suggests that the system will likely perform best on queries related to worship and core beliefs due to the high density of supporting documents.

Chapter 5 System Analysis & Design

1. System initial analysis

This section presents the initial analysis and key features identified during the early stages of project planning and development.

System Requirements:

*Category: Supervisor Agent (3 requirements)

Req_ID	Requirement Title	Description	Priority
FR-SA-01	Supervisor Agent	The system shall implement a Supervisor Agent that serves as the central orchestrator for the entire query lifecycle. It is responsible for managing the AgentState and coordinating the sequential execution of specialized agents by utilizing its assigned tools.	4
FR-SA-02	Agent Task Assignment Tool	The Supervisor Agent shall include a task assignment tool that determines which specialized agents to activate based on query type, current workflow state, and previous agent outputs.	4
FR-SA-03	Iteration Control Tool	The Supervisor Agent shall include an iteration control tool that tracks retrieval cycle counts, enforces maximum iteration limits, and decides whether to continue iterating or proceed to final answer generation.	4

Table 2 supervisor agent requirements

Category: Query Analysis Agent (7 requirements)

Req_ID	Requirement Title	Description	Priority
FR-QAA-04	Query Analysis Agent	The system shall implement a Query Analysis Agent responsible for decomposing and understanding the user's raw input. It transforms this input into a structured, actionable plan for the retrieval and verification agents by utilizing its assigned tools.	1
FR-QAA-05	Typo Correction Tool	The Query Analysis Agent shall include a typo correction tool that detects and corrects common Arabic spelling errors, character substitutions, and diacritic mistakes before query processing.	2
FR-QAA-06	Query Normalization Tool	The Query Analysis Agent shall include a query normalization tool that standardizes diacritics, unifies letter variants, removes excessive whitespace, and handles right-to-left text encoding.	2
FR-QAA-07	Intent Classification Tool	The Query Analysis Agent shall include an intent classification tool that categorizes queries into types including thematic search, specific hadith lookup, narrator-based search, comparative analysis, or multi-part questions.	1

FR-QAA-08	Query Decomposition Tool	The Query Analysis Agent shall include a query decomposition tool that splits multi-part questions into separate sub-queries while maintaining relationships between them for coordinated processing.	2
FR-QAA-09	Collection Target Detection Tool	The Query Analysis Agent shall include a collection detection tool that identifies whether the query targets Ṣaḥīḥ al-Bukhārī, Ṣaḥīḥ Muslim, both collections, or is collection-agnostic.	2
FRQ-AA-10	Input Source Identification Tool	The Query Analysis Agent shall include an input source identification tool that detects whether the user is querying base knowledge (JSON collections), providing custom hadith text, or uploading additional documents for context-specific retrieval.	1

Table 3 query analysis agent requirements

Category: Retrieval Agent (9 requirements)

Req_ID	Requirement Title	Description	Priority
FR-RA-11	Retrieval Planning Agent	The system shall implement a Retrieval Agent responsible for executing search strategies against the knowledge base. Its purpose is to locate and fetch relevant documents (Hadiths) based on the current query plan by utilizing its assigned tools.	1

FR-RA-12	Semantic Search Tool	The Retrieval Agent shall include a semantic search tool that performs vector similarity searches against embedded hadith chunks using cosine similarity or equivalent distance metrics.	1
FR-RA-13	Keyword Search Tool	The Retrieval Agent shall include a keyword search tool that performs lexical matching using BM25 or similar algorithms to retrieve hadiths containing exact terms or morphological variants.	2
FR-RA-14	Metadata Filter Tool	The Retrieval Agent shall include a metadata filtering tool that narrows results based on book names, chapter titles, hadith numbers, narrator names, or collection identifiers.	2
FR-RA-15	Hybrid Search Tool	The Retrieval Agent shall include a hybrid search tool that combines semantic and keyword-based retrieval, fusing results with score normalization and deduplication.	2
FR-RA-16	Result Aggregation Tool	The Retrieval Agent shall include a result aggregation tool that combines outputs from multiple retrieval operations and across cycles, applies deduplication, and produces unified ranked result sets.	2
FR-RA-17	User-Provided Hadith Processor Tool	The Retrieval Agent shall include a user-provided hadith processor tool that temporarily indexes and searches hadith texts submitted directly by the user within the query context.	2

FR-RA-18	Document Ingestion Tool	The Retrieval Agent shall include a document ingestion tool that processes user-uploaded documents, extracts relevant text, generates temporary embeddings, and enables retrieval from uploaded content alongside base knowledge.	2
FR-RA-19	Document Reassembly	The system shall detect if a retrieved hadith is a chunk of a larger hadith and then fetch all sibling chunks from the database, concatenate them into a single hadith	1
FR-RA-20	Find chapter tool	A specialized lookup utility designed to map specific subjects or topics to their corresponding chapter identifiers within major Hadith collections.	2

Table 4 retrieval agent requirements

Category: Evaluation Agent (6 requirements)

Req_ID	Requirement Title	Description	Priority
FR-EA-21	Evaluation Agent	The system shall implement an Evaluation Agent responsible for assessing the quality, relevance, and faithfulness of information at key stages of the workflow. It provides critical feedback to the Supervisor by utilizing its assigned tools.	1

FR-EA-22	Quality Assessment Tool	The Evaluation Agent shall include a quality assessment tool that analyzes semantic relevance, coverage completeness, source diversity, and alignment with query intent for retrieved results.	1
FR-EA-23	Gap Identification Tool	The Evaluation Agent shall include a gap identification tool that detects missing information, unanswered query aspects, or insufficient evidence in current retrieval results.	2
FR-EA-24	Confidence Scoring Tool	The Evaluation Agent shall include a confidence scoring tool that computes numeric confidence values based on similarity scores, result consistency, and information completeness.	2
FR-EA-25	Stopping Condition Tool	The Evaluation Agent shall include a stopping condition tool that autonomously determines when sufficient information has been gathered or when additional retrieval would yield diminishing returns.	1
FR-EA-26	Grounding Validation Tool	The Evaluation Agent shall include a grounding validation tool that verifies all information candidates for answer generation exist verbatim in retrieved	1

		hadith sources, rejecting any content not directly traceable to the knowledge base.	
--	--	---	--

Table 5 evaluation agent requirements

***Category: Answer Synthesis Agent (6 requirements)**

Req_ID	Requirement Title	Description	Priority
FR-ASA-27	Answer Synthesis Agent	The system shall implement an Answer Synthesis Agent responsible for generating a final, coherent, and grounded answer for the user. It transforms raw retrieved data into a natural language response by utilizing its assigned tools.	1
FR-ASA-28	Information Combination Tool	The Answer Synthesis Agent shall include an information combination tool that synthesizes content from multiple hadiths across retrieval cycles into unified coherent narratives.	1
FR-ASA-29	Citation Generation Tool	The Answer Synthesis Agent shall include a citation generation tool that produces complete references including collection	1

		name, book name, chapter title, hadith number, and narrator chain for all cited hadiths.	
FR-ASA-30	Grounding Enforcement Tool	The Answer Synthesis Agent shall include a grounding enforcement tool that ensures every claim in generated answers is traceable to specific retrieved hadith texts without hallucination or fabrication.	1
FR-ASA-31	Answer Formatting Tool	The Answer Synthesis Agent shall include an answer formatting tool that structures responses appropriately for query type, distinguishes direct quotes from explanations, and maintains Arabic linguistic accuracy.	2
FR-ASA-32	No-Answer Detection Tool	The Answer Synthesis Agent shall include a no-answer detection tool that identifies when retrieved information is insufficient to answer the query, returning explicit "cannot answer" responses rather than generating speculative content.	1

Table 6 answer synthesis agent requirements

Category: Data Ingestion Component (5 requirements)

Req_ID	Requirement Title	Description	Priority
FR-DIC-33	Data Ingestion Pipeline	The system shall implement a Data Ingestion Pipeline responsible for loading hadith datasets from JSON files, extracting metadata, generating embeddings, and populating vector databases with searchable indices.	1
FR-DIC-34	JSON Parsing	The Data Ingestion Pipeline shall include a JSON parsing tool that extracts hadith text, narrator chains, book references, chapter references, and hadith numbers from structured JSON datasets	1
FR-DIC-35	Text Preprocessing	The Data Ingestion Pipeline shall include a text preprocessing tool that normalizes Arabic text, handles diacritics, removes non-Arabic artifacts, and standardizes formatting during ingestion.	1
FR-DIC-36	Embedding Generation	The Data Ingestion Pipeline shall include an embedding generation tool that converts preprocessed hadith chunks into dense vector representations using Arabic-optimized embedding models from HuggingFace.	1

FR-DIC-37	Vector Storage	The Data Ingestion Pipeline shall include a vector storage tool that persists embeddings along with associated text and metadata in vector databases optimized for semantic similarity search.	1
-----------	----------------	--	---

Table 7 data ingestion component requirements

Category: Infrastructure & Framework Integration (4 requirements)

Req_ID	Requirement Title	Description	Priority
FR-FI-38	External AI Service Integration	The system shall integrate with externally-hosted, serverless AI endpoints on Modal.com for all heavyweight GPU tasks, including model reasoning and embedding generation	1
FR-FI-39	LangGraph Workflow Engine	The system shall utilize LangGraph as the core workflow orchestration engine, implementing stateful multi-agent graphs with conditional routing, cyclic execution patterns, and shared state management across all agents.	1
FR-FI-40	Agent State Management	The system shall maintain a shared state object accessible to all agents through LangGraph, preserving query context, retrieval results, evaluation feedback, and	1

		intermediate reasoning across workflow execution.	
FR-FI-41	LangSmith Observability Integration	The system shall integrate LangSmith for observability and tracing, logging agent decisions, retrieval cycles, LLM calls, token usage, execution times, and workflow paths to enable debugging and performance monitoring.	2
FR-INF-42	Generative Reasoning Engine	A high-performance interface for Large Language Models that handles natural language understanding, context synthesis, and response generation. It serves as the primary intelligence layer for complex task execution.	1
FR-INF-43	Vector Encoding Service	Converts unstructured text into high-dimensional numerical vectors. This service enables semantic similarity mapping, allowing the system to "understand" the conceptual relationship between different pieces of data.	1

FR-INF-44	Contextual Relevance Optimizer	A secondary scoring layer that evaluates a subset of retrieved documents against the user query. It re-orders results based on deep semantic relevance to ensure the most accurate information is prioritized.	2
-----------	--------------------------------------	--	---

Table 8 infrastructure & framework integration requirements

Category: User Interface (4 requirements)

Req_ID	Requirement Title	Description	Priority
FR-UI-45	Query History View	The system shall provide a user interface that displays a chronological list of past query sessions, allowing users to revisit previous questions and answers.	2
FR-UI-46	Reasoning Trace Display	The system shall include an option for users to view a simplified, human-readable version of the agent reasoning chain (e.g., "Searching Bukhari...", "Evaluating results...") for transparency.	2
FR-UI-47	Relevance Feedback Tool	The system shall include a feedback mechanism (e.g., thumbs up/down buttons) on final answers, allowing users to rate response quality for future system improvements.	2
FR-UI-48	Read Answer	The system shall present all final answers to the user in clear, coherent, and grammatically correct, ensuring the text is easily understandable by a non-technical audience.	1

Table 9 user interface requirements

***Category: System Administration (3 requirements)**

Req_ID	Requirement Title	Description	Priority
FR-ADM-49	Knowledge Base Management Dashboard	The system shall provide an administrative dashboard to trigger new data ingestion batches, view indexing status, and manually remove obsolete records from the vector database.	3
FR-ADM-50	Dynamic Configuration Interface	The system shall provide an administrative interface to adjust key agent parameters at runtime (e.g., maximum iteration limits, similarity score thresholds) without requiring a full system restart.	3
FR-ADM-51	System Health Monitoring View	The system shall provide an aggregated view of key operational metrics (e.g., average response time, agent failure rates, daily query count) for real-time system monitoring.	2

Table 10 system administration requirements

Category: Non-Functional Requirements (10 requirements)

Req_ID	Requirement Title	Description	Priority
*NFR-01	Concurrent User Support	The system shall support at least 50 concurrent users submitting queries simultaneously without degradation in response time exceeding 20% of baseline single-user performance. The system shall implement request queuing and load	2

		balancing to handle traffic spikes up to 100 concurrent users.	
*NFR-02	Token Efficiency	The system shall optimize LLM token usage to consume no more than 10,000 tokens per query on average (including all agent reasoning, tool calls, and answer generation across all retrieval cycles), ensuring cost-effective operation at scale.	2
*NFR-03	System Availability	The system's overall availability is dependent on the 99.5% uptime of the main application server <i>and</i> the service-level agreement (SLA) of the Modal.com API.	1
*NFR-04	Graceful Degradation	The system shall implement fallback mechanisms ensuring that if any single agent fails, the system returns a partial response or informative error message within 10 seconds rather than hanging indefinitely. Critical agent failures (Retrieval, Synthesis) shall trigger automatic retries (maximum 3 attempts) before returning error responses.	1

NFR-05	Data Consistency	The system shall ensure 100% consistency between retrieved hadith texts and their source metadata (book, chapter, hadith number, narrator chain) across all operations, with automated validation checks during data ingestion and periodic integrity audits every 24 hours detecting any discrepancies.	1
*NFR-06	Rate Limiting and Abuse Prevention	The system shall enforce rate limiting of 100 queries per hour per authenticated user and 20 queries per hour per IP address for unauthenticated access, preventing denial-of-service attacks and resource exhaustion. The system shall implement exponential backoff for clients exceeding limits and temporarily block IP addresses showing abusive patterns (10+ violations within 1 hour).	1
*NFR-07	Configuration Management	The system shall externalize all configuration parameters (model names, temperature settings, retrieval thresholds, iteration limits, API endpoints, API endpoints for Modal, and Modal API keys) through environment variables or configuration files, enabling deployment-specific customization without code changes. Configuration changes shall not	2

		require system restarts except for critical parameters (database connections, authentication keys).	
NFR-08	Monitoring and Observability	The system shall expose comprehensive monitoring metrics including agent execution times, retrieval cycle counts, token usage per query, error rates per agent, and API response times through standard monitoring interfaces (Prometheus metrics endpoint). The system shall integrate with LangSmith or equivalent observability platforms providing full tracing of agent decisions and workflow execution paths.	2
*NFR-09	Error Logging and Diagnostics	The system shall implement structured logging (JSON format) with severity levels (DEBUG, INFO, WARNING, ERROR, CRITICAL) capturing all agent decisions, retrieval operations, LLM calls, and system errors with sufficient context (query ID, agent name, timestamp, stack traces) to enable root cause analysis. Logs shall be retained for 90 days and include correlation IDs linking all operations within a single query workflow.	2

NFR-10	Cold Start Response Time	On the first request to an idle endpoint, the system shall return a response within 90 seconds	2
NFR-11	Warm Start Response Tim	For subsequent requests, the system shall return a response to a standard query in under one minute.	1
NFR-12	Operational Cost Model	The system's operational cost for AI processing shall be variable (pay-as-you-go), based on per-second GPU usage from the Modal.com service.	2
NFR-13	Data in Transit Security	All communication between the main application server and the Modal AI endpoints must be encrypted using HTTPS.	1

Table 11 non-functional requirements

* *means that these requirements will be build in the future*

2. Requirements Modeling

Use case diagram

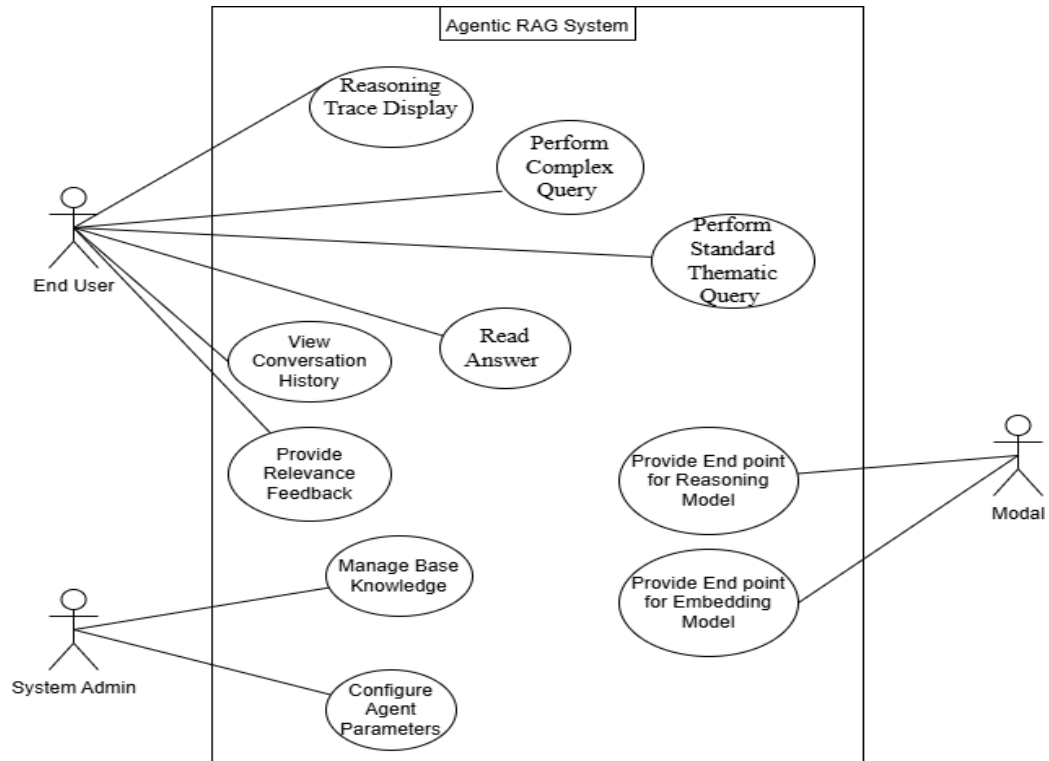


Figure 11 use case diagram

Class Diagram

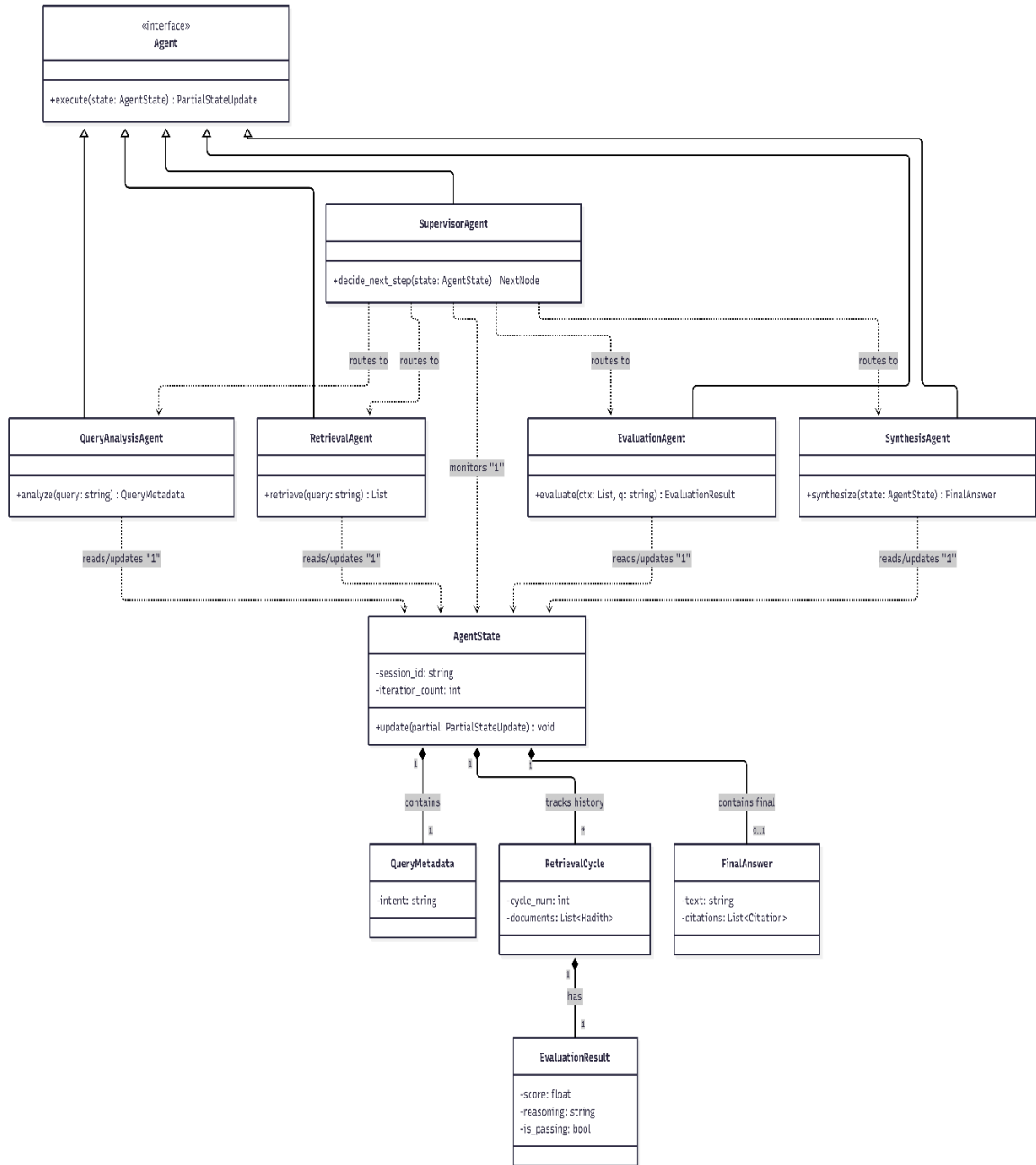


Figure 12 class diagram

3. Test Cases Definition

Test Case Scenario:		Sce-01: Query Analysis Agent Functionality.	
Test Case ID	Test Case Title	Test Steps	Expected Result
TC-QAA-04	Verify Query Analysis Agent Orchestration	1. Launch the system. 2. Input a complex query (e.g., "Wudu steps and prayer times"). 3. Monitor the agent's log/state.	The agent should accept the raw input and output a structured "Search Plan" object containing decomposed queries and metadata.
TC-QAA-05	Verify Typo Correction Tool	1. Input a query with deliberate Arabic typos (e.g., "الصلاة" instead of "الصلوات"). 2. Execute the analysis phase. 3. Check the corrected_query in the state.	The system detects the typo and updates the query string to the correct spelling ("الصلوة") before processing.
TC-QAA-06	Verify Query Normalization Tool	1. Input a query with mixed diacritics and extra spaces (e.g., "كِتَابُ الْإِيمَانِ"). 2. Execute normalization. 3. Inspect the processed text.	Diacritics are removed, alef forms are unified (to "ا"), and extra whitespace is trimmed (e.g., "كِتَابُ الْإِيمَانِ").
TC-QAA-07	Verify Intent Classification Tool	1. Input queries representing different intents (e.g., "Show me hadiths by Abu Hurairah")	The first is classified as narrator_search, and the second as

		vs. "How to pray"). 2. Check the intent field in the output.	thematic_search or procedural_query.
TC-QAA-08	Verify Query Decomposition Tool	1. Input a multi-part question: "What is the ruling on fasting and what breaks it?". 2. Review the sub_queries list in the agent state.	The system splits the input into two distinct sub-queries: 1) "Ruling on fasting", 2) "Invalidators of fasting".
TC-QAA-09	Verify Collection Target Detection Tool	1. Input a query explicitly mentioning a source: "Hadith about intention in Bukhari". 2. Check the target_collection metadata.	The system identifies and sets the target collection strictly to sahih_bukhari.
TC-QAA-10	Verify Input Source Identification Tool	1. Paste a specific text block into the context window and ask "Explain this". 2. Check the input_source classification.	The system identifies the source as user_provided_text rather than querying the database.

Test Case Scenario:		Sce-02: Retrieval Agent Functionality.	
Test Case ID	Test Case Title	Test Steps	Expected Result
TC-RA-11	Verify Retrieval Planning Execution	1. Provide a search_plan with 3 sub-queries to the Retrieval Agent. 2. Trigger the retrieval process.	The agent executes searches for all 3 sub-queries sequentially or in parallel and returns results for each.
TC-RA-12	Verify Semantic Search Tool	1. Search using a conceptual query without exact keywords (e.g., "cleanliness ritual"). 2. Review the retrieved Hadith text.	Results include Hadiths containing "Wudu" or "Ghusl" (ablution), confirming vector similarity matched the concept.
TC-RA-13	Verify Keyword Search Tool	1. Search for a specific unique term (e.g., "Yarmouk"). 2. Review results.	The system returns only Hadiths containing the exact word "Yarmouk" using BM25/lexical matching.
TC-RA-14	Verify Metadata Filter Tool	1. Set a filter for narrator="Aisha" and query "Fasting". 2. Inspect the metadata of the returned results.	All returned Hadiths must have narrator field equal to "Aisha".
TC-RA-15	Verify Hybrid Search Tool	1. Enter a query that benefits from both exact match and context (e.g., "Ruling on	The system combines results from Vector Search and Keyword

		Zakat for camels"). 2. Check the retrieval logs.	Search, ranking them by a fused score.
TC-RA-16	Verify Result Aggregation Tool	1. Perform a search that returns overlapping results from Semantic and Keyword tools. 2. Check the final list presented to the Evaluation Agent.	The list is deduplicated (no repeated Hadith IDs) and sorted by the highest combined relevance score.
TC-RA-17	Verify User-Provided Hadith Processor	1. User uploads/pastes a custom text text. 2. User asks a question about that specific text. 3. Execute search.	The system retrieves answers <i>only</i> from the temporary index of the provided text, ignoring the main DB if instructed.
TC-RA-18	Verify Document Ingestion Tool (Runtime)	1. Upload a PDF/Text file during the session. 2. Query for unique content inside that file.	The system successfully finds and retrieves the specific sentence/fact from the uploaded file.
TC-RA-19	Verify Document Reassembly	1. Search for a long Hadith known to be split into chunks (e.g., Hadith of Ka'b bin Malik). 2. View the result details.	The system detects it is a chunk, fetches all sibling chunks, and displays the full, concatenated Hadith text.
TC-RA-20	Verify Find Chapter Tool	1. Query for the topic "Inheritance" (Al-Fara'id). 2. Check the retrieval output.	The system returns the specific Chapter_ID or Chapter Title

			corresponding to "Inheritance" in the books.
--	--	--	--

Test Case Scenario: Sce-03: Evaluation Agent Functionality.			
Test Case ID	Test Case Title	Test Steps	Expected Result
TC-EA-21	Verify Evaluation Agent Logic	1. Feed the agent a query and a set of results. 2. Run the evaluation cycle.	The agent outputs a structured evaluation object containing status (PASS/FAIL) and reasoning.
TC-EA-22	Verify Quality Assessment Tool	1. Provide a query about "Prayer" and irrelevant docs about "Trade". 2. Check the quality score.	The tool returns a low relevance score (e.g., < 0.5) and marks the results as irrelevant.
TC-EA-23	Verify Gap Identification Tool	1. Provide a query "What are the 5 pillars?" but only retrieve docs for 3 pillars. 2. Check the missing_info output.	The tool explicitly lists the missing pillars as "Gaps" that need to be filled.

TC-EA-24	Verify Confidence Scoring Tool	1. Provide high-quality exact matches. 2. Check the confidence metric.	The tool calculates a high confidence score (e.g., > 0.85) based on semantic similarity and coverage.
TC-EA-25	Verify Stopping Condition Tool	1. Simulate a state where confidence_score > threshold. 2. Check the workflow decision.	The agent signals STOP / GENERATE, ending the retrieval loop.
TC-EA-26	Verify Grounding Validation Tool	1. Generate a draft answer containing a fact NOT in the retrieved docs. 2. Run validation.	The tool flags the ungrounded statement as a hallucination or removes it from the final answer.

Test Case Scenario:		Sce-04: Data Ingestion Functionality.	
Test Case ID	Test Case Title	Test Steps	Expected Result
TC-DIC-33	Verify Data Ingestion Pipeline	1. Place a raw JSON dataset in the source folder. 2. Run the ingestion script. 3. Query the vector DB.	The pipeline runs without errors, and the new data is searchable in the vector database.

TC-DIC-34	Verify JSON Parsing	1. Input a JSON file with specific fields (text, hadith_no, book). 2. Run the parser.	The parser correctly extracts and maps all fields to the internal data schema without data loss.
TC-DIC-35	Verify Text Preprocessing	1. Input raw Arabic text with HTML tags or special chars. 2. Run the preprocessor.	Output is clean plain text, with normalized characters and no artifacts.
TC-DIC-36	Verify Embedding Generation	1. Input a sample text chunk. 2. Call the embedding service. 3. Check output format.	The system returns a dense vector (list of floats) of the correct dimension (e.g., 1024 or 768).
TC-DIC-37	Verify Vector Storage	1. Ingest a specific Hadith. 2. Search for its ID in ChromaDB/Vector Store.	The record exists, containing both the embedding vector and the correct metadata payload.

Test Case Scenario: Sce-05: Infrastructure & Framework Integration Functionality.			
Test Case ID	Test Case Title	Test Steps	Expected Result
TC-FI-38	Verify External AI Service Integration	1. Trigger a function requiring heavy computation (embedding/LLM). 2. Monitor network traffic/logs.	The request is successfully sent to the external endpoint (Modal.com) and a valid response is received.
TC-FI-39	Verify LangGraph Workflow Engine	1. Initiate a user query. 2. Trace the execution path.	The system moves through the defined nodes (Start → Analysis → Retrieval → Eval) following the graph logic.
TC-FI-40	Verify Agent State Management	1. Perform a multi-step query. 2. Check the state object at step 3.	The state contains data preserved from step 1 (e.g., original query, initial plan) and step 2 (retrieved docs).
TC-FI-41	Verify LangSmith Observability	1. Run a full workflow cycle. 2. Open LangSmith dashboard.	A trace for the session appears, showing inputs/outputs for every agent step and LLM call.

TC-INF-42	Verify Generative Reasoning Engine	1. Send a complex prompt requiring synthesis to the LLM. 2. Check the response.	The LLM generates a coherent, logical text response adhering to the prompt instructions.
TC-INF-43	Verify Vector Encoding Service	1. Send a batch of 10 texts to the encoder. 2. Check output.	The service returns a list of 10 distinct vectors corresponding to the inputs.
TC-INF-44	Verify Contextual Relevance Optimizer	1. Provide a list of 10 mixed-relevance docs. 2. Run the re-ranker/optimizer.	The list is re-ordered so that the most relevant documents are at indices 0-4.

Test Case Scenario: Sce-06: User Interface Functionality.			
Test Case ID	Test Case Title	Test Steps	Expected Result
TC-UI-48	Verify Readability of Final Answer	1. Ask a question in Arabic. 2. Wait for the final response. 3. Review the displayed text.	The answer is displayed in clear, right-to-left (RTL) aligned Arabic, with proper formatting (paragraphs/bullet points) and is grammatically correct.

Table 12 test cases definition

4. System Architecture

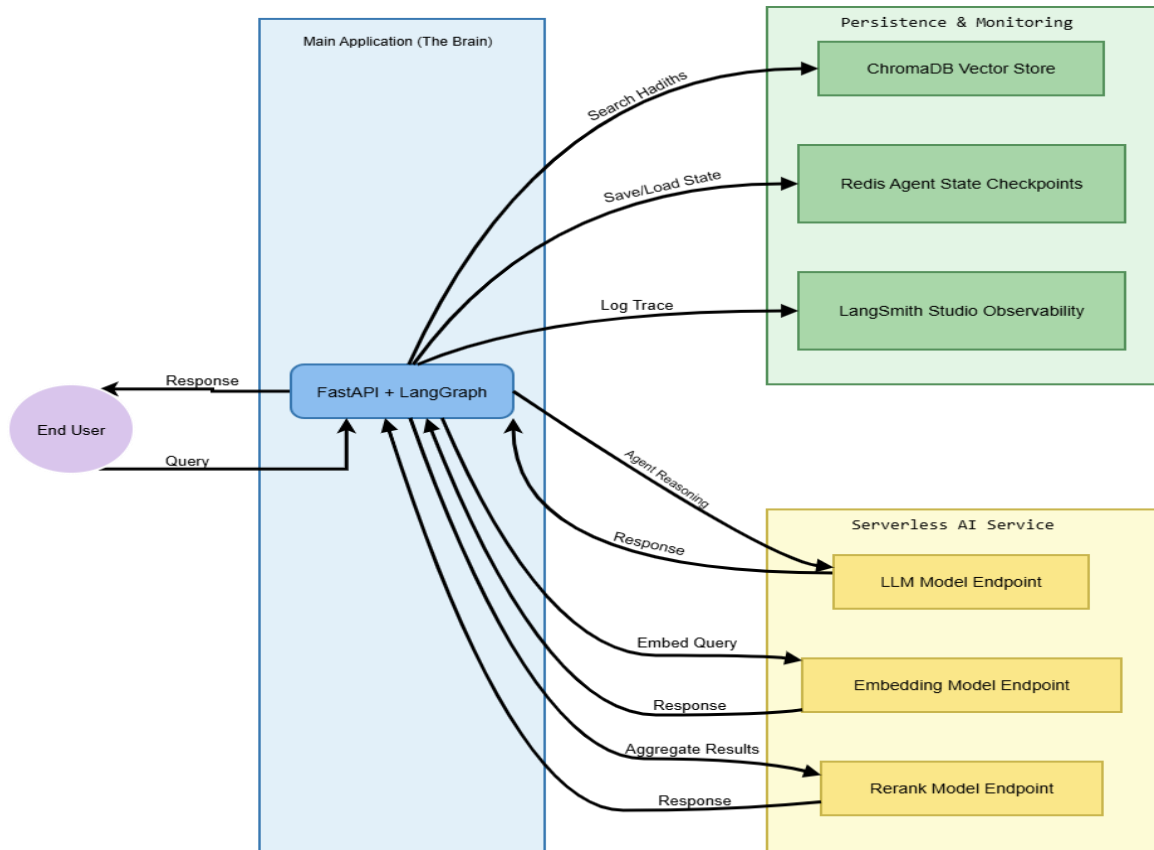


Figure 13 system architecture

The system uses a third-party service powered by Modal for serverless inference and **LangGraph** for stateful orchestration. The architecture is divided into three main layers:

1. The Interface Layer (Client): A Streamlit/FastAPI application that captures user intent and displays citations.
2. The Orchestration Layer (Brain): Built on LangGraph, this layer manages the conversation state (AgentState). It is not a linear chain but a cyclical graph that can loop, self-correct, and route queries between specialized agents.
3. The Tool/Data Layer (Muscle): Contains the specialized execution units:
 - Vector Store: ChromaDB hosting the BGE-M3 embeddings.

- Search Engine: The Hybrid Search (BM25 + Semantic) module.
- Inference Engine: Qwen-2.5-14B hosted on high-performance GPUs via Modal.

5. State Graph Design (The Core Logic)

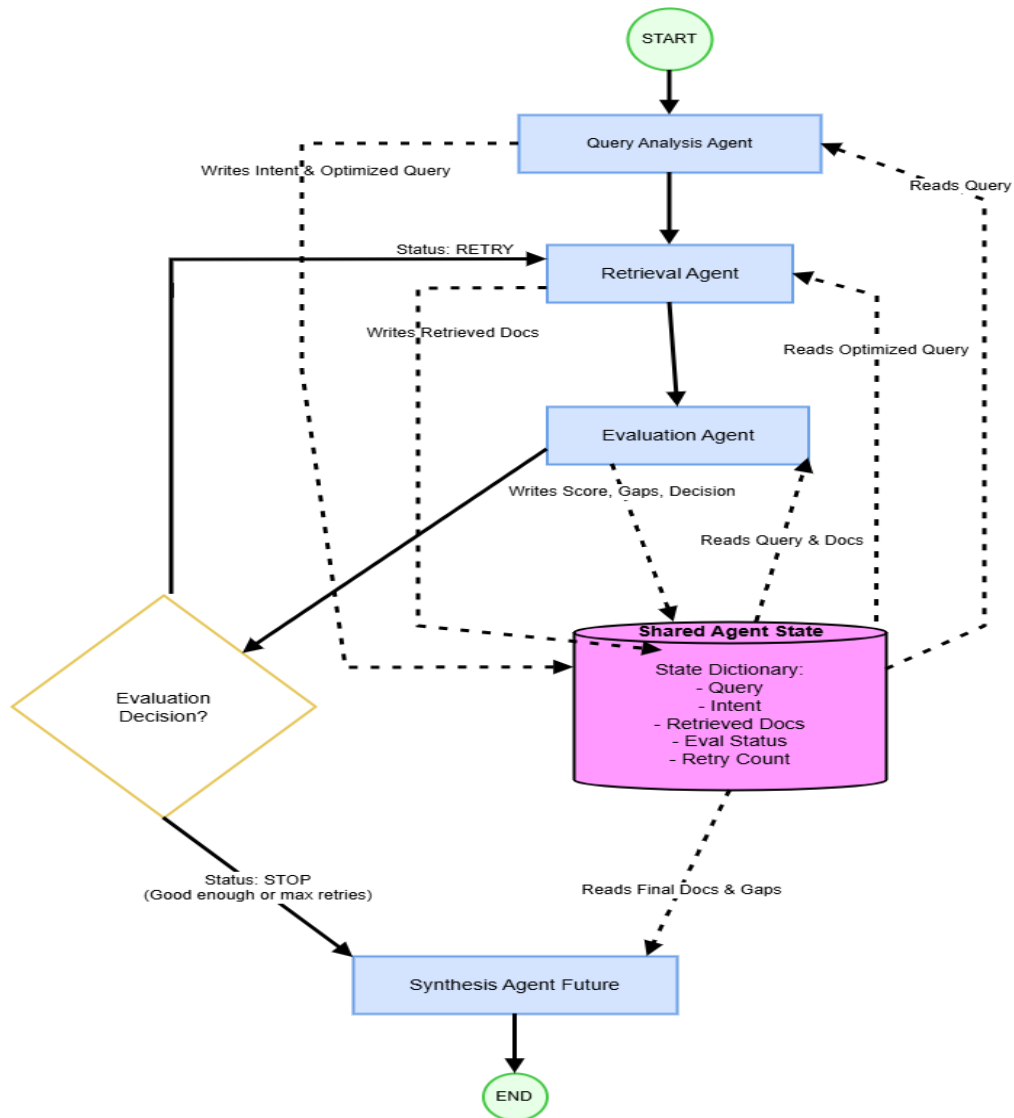


Figure 14 state graph design

Unlike traditional state machines, the system uses a shared state schema that passes between nodes.

5.1 The State Schema

The system maintains a typed dictionary `AgentState` defined in `src/graph/state.py` which tracks the lifecycle of a query:

- `messages`: A list of chat history (User, AI, Tool outputs).
- `query`: The current refined search query.
- `documents`: A list of retrieved Document objects found so far.
- `intent`: The classified intent (e.g., `specific_lookup`, `thematic_search`).
- `attempts`: A counter to prevent infinite retrieval loops.

5.2 Graph Nodes & Flow

The workflow is designed as a directed graph where:

1. **Start** → **Query Analysis Node**: Normalizes the query and detects intent.
2. **Router**: Decides whether to go to Retrieval (for knowledge gaps) or Direct Answer (for chitchat).
3. **Retrieval Node**: Executes the RetrievalAgent loop.
4. **Grade/Evaluate Node**: Checks if the retrieved documents answer the query.
 - a. If Yes → **Generate Node** (Synthesize answer).
 - b. If No → **Rewrite Query** → **Retrieval Node** (Loop back).

6. Multi-Agent Design

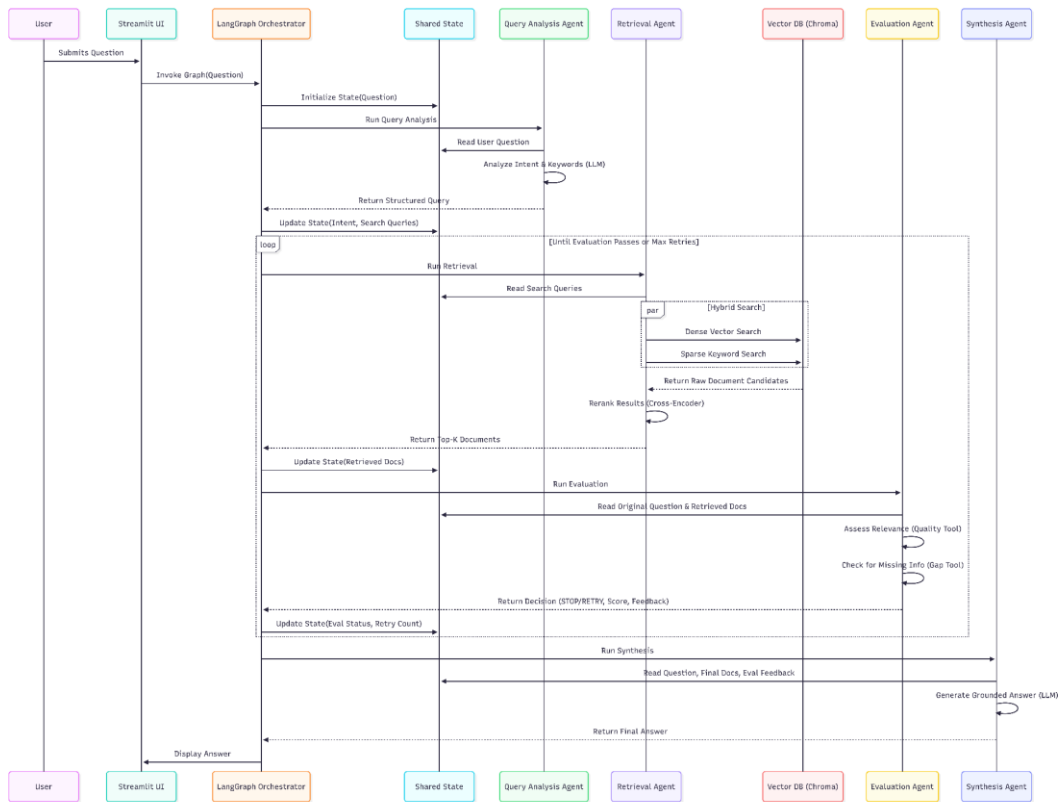


Figure 15 multi-agent design

The system utilizes specialized agents with distinct responsibilities:

A. Query Analysis Agent

- Role: Pre-processor.
- Logic: It uses a few-shot classification prompt to categorize queries. For example, "Longest Hadith" is classified as a `metadata_query`, while "Virtues of Fasting" is a `thematic_search`. It also performs query decomposition for complex questions.

B. Retrieval Agent (Autonomous)

- Role: The researcher.
- Design: Implements the ReAct (Reason + Act) pattern. It does not simply search once; it iteratively:

1. Thinks: "I need to find the chapter ID for 'Prayer' first."
2. Acts: Calls find_chapter tool.
3. Observes: Result is "Chapter 8".
4. Thinks: "Now I will search for hadiths in Chapter 8."
5. Acts: Calls hybrid_search tool.

7. Search & Retrieval Strategy (Hybrid Design)

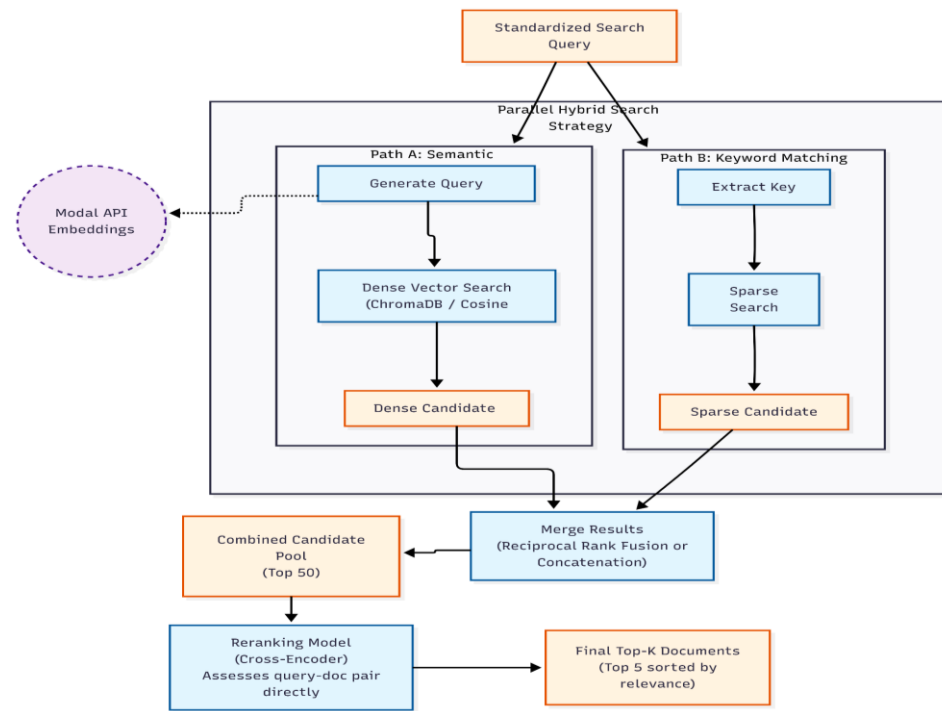


Figure 16 hybrid search flow

To address the limitations of pure vector search, a Hybrid Search Strategy was designed using Reciprocal Rank Fusion (RRF):

1. Keyword Branch (BM25):

- Utilizes the rank_bm25 algorithm.
- Effective for exact matches like "Hadith 1" or narrator names ("Abu Hurairah").
- Running on a separate ProcessPoolExecutor to avoid Python GIL blocking.

2. Semantic Branch (Vector):

- Utilizes BAAI/bge-m3 embeddings stored in ChromaDB.
- Captures conceptual similarity (e.g., matching "salah" with "prayer").

3. Fusion (RRF):

- The results are merged using the formula:
$$\text{Score} = \frac{l}{k + \text{rerank}_{\text{semantic}}} + \frac{l}{k + \text{rerank}_{\text{keyword}}}$$
 - This ensures that a document appearing in both lists (even with different rankings) is boosted to the top.

8. Data Modeling & Schema Design

The system architecture utilizes a dual-schema approach: a Persistent Schema for the vector database (ChromaDB) to handle long-term knowledge storage, and an Ephemeral State Schema (LangGraph) to manage the runtime context of the agentic workflow.

8.1 Persistent Data Schema (Vector Store)

The system uses ChromaDB as a hybrid storage engine. Unlike traditional relational databases, the schema is designed around Semantic Chunks. Each record consists of three core components (ID, Document, Embedding) accompanied by a rich metadata dictionary used for pre-filtering and post-retrieval reassembly

Component	Field Name	Data Type	Description	Usage
Core	id	String	Unique key (e.g., bukhari_1_15_ar_0).	Primary Key.
	document	String	The plain text segment of the Hadith.	Keyword Search (BM25) & Display.
	embedding	List[Float]	1024-dimensional dense vector (BGE-M3).	Semantic Similarity Calculation.
Relational	parent_hadith_id	String	ID of the full source Hadith.	Reassembly: Links chunks back to the whole text.
	chunk_index	Integer	Sequence number (0, 1, 2...).	Ordering segments during display.
	is_chunked	Boolean	Flag indicating if text was split.	Triggers reassembly logic if True.
Metadata	collection	String	Source name (e.g., "sahih_bukhari").	Hard Filtering: Restricts search scope.

	narrator	String	Name of the primary narrator.	Metadata Filtering (e.g., "Narrated by Aisha").
	chapter_id	Integer	The theological chapter ID.	Context grouping.
	language	String	arabic or english.	Cross-lingual routing.

Table 13 Vector Database Document Model

Design Decision (Flattened Metadata): The hierarchical JSON structure of the source books was "flattened" into this metadata schema. This allows the Retrieval Agent to execute complex SQL-like filters (e.g., WHERE narrator='Abu Hurairah' AND collection='Muslim') directly within the vector database query engine.

8.2 Data Transformation Pipeline

The data undergoes specific transformations to bridge the gap between raw text and actionable intelligence:

1. **Ingestion:** Raw JSON → Text Cleaning → Semantic Chunking (max 512 tokens).
2. **Vectorization:** Chunk Text → BGE-M3 Model → Dense Vector (1024-d).
3. **Retrieval:** Query Vector → Nearest Neighbor Search → Top-K Chunks.
4. **Synthesis:** Top-K Chunks → Reassembly Logic → Full Context Window.

9. Used Design Patterns:

Strategy Design Pattern – Search Execution

Description: The Strategy Pattern is used to define a family of search algorithms (Vector, Keyword, Hybrid), encapsulate them, and make them interchangeable. The RetrievalAgent determines which strategy to execute based on the user's intent at runtime.

The use of the Strategy pattern improves the system:

- **Dynamic Behavior:** The system can switch between "Exact Keyword Search" (for names) and "Semantic Search" (for concepts) dynamically without changing the agent's core code.
- **Extensibility:** If you want to add a new search method (e.g., "Graph Search"), you simply add a new strategy method without rewriting the orchestrator.
- **Separation of Concerns:** The logic for deciding what to do is separated from the logic of how to do the search.

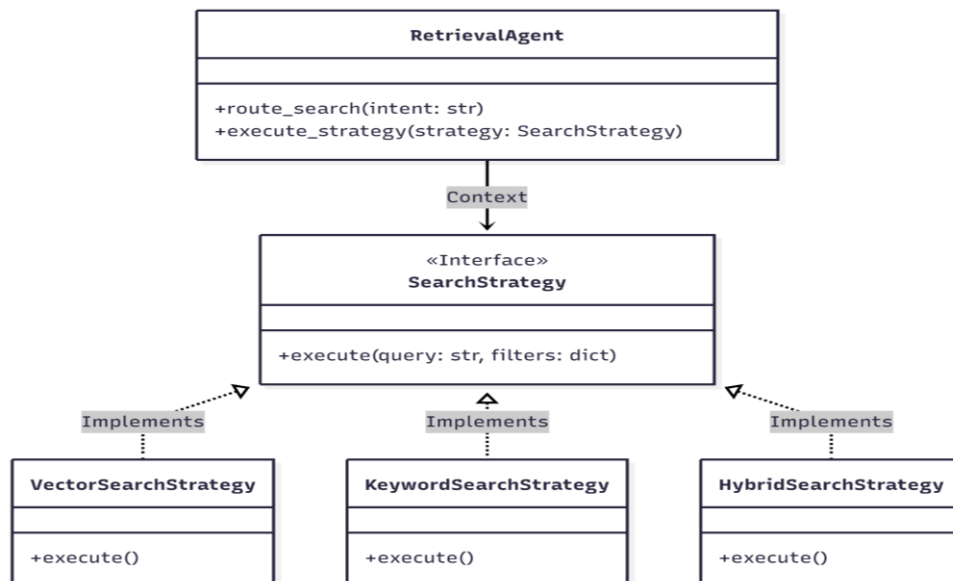


Figure 17 strategy pattern

Singleton Design Pattern – Infrastructure Management

Description: The Singleton Pattern ensures that a class has only one instance and provides a global point of access to it. In the system, it is implemented to manage resource-heavy connections like the LLM Client and Database Client.

The use of the Singleton pattern improves the system:

- **Resource Management:** Prevents the system from opening thousands of redundant connections to ChromaDB or the Modal API, which would crash the application or exhaust memory.
- **Global Access:** Ensures that every agent (Query Analysis, Retrieval, Eval) uses the exact same configuration and connection pool.
- **Performance:** Initialization (which is expensive) happens only once at startup.

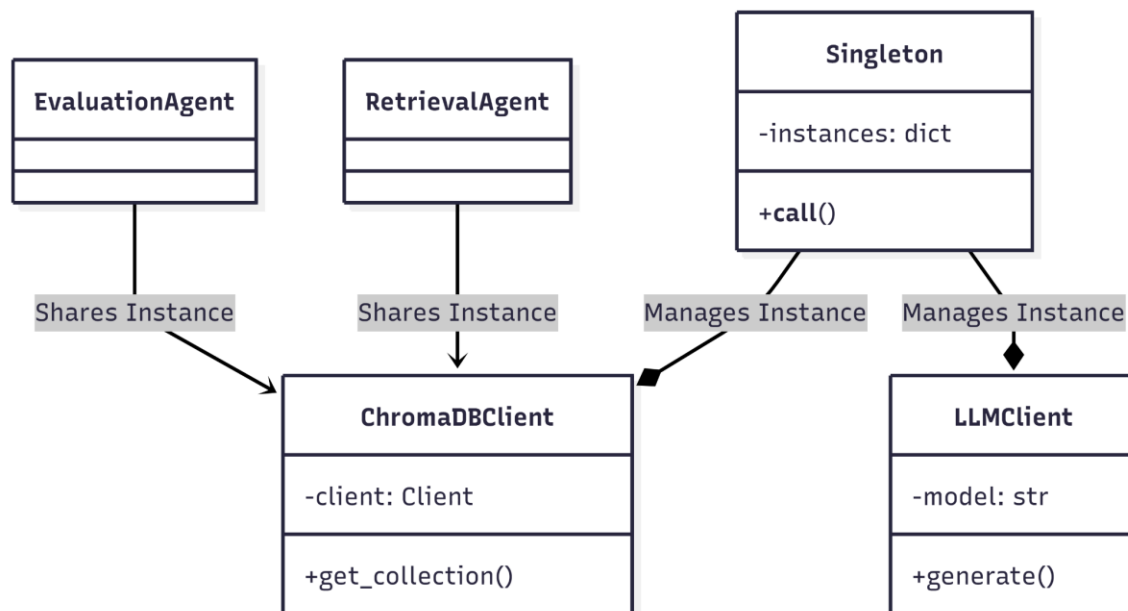


Figure 18 single pattern

State Design Pattern

Description: The State Pattern allows an object to alter its behavior when its internal state changes. In the system, using LangGraph, the entire application flows through a shared AgentState. The "Workflow" acts as the State Machine, transitioning the system from "Analysis" to "Retrieval" to "Evaluation" based on the current data in the state.

The use of the State pattern improves the system:

- **Complex Workflow Management:** Allows for cyclic workflows (Loops). For example, if the state is status="retry", the system transitions back to Retrieval instead of End.
- **Context Preservation:** The AgentState object persists memory (query history, retrieved docs) across different stateless agents.
- **Predictability:** The flow of the application is explicitly defined as a graph of nodes and edges, making debugging easier compared to spaghetti code.

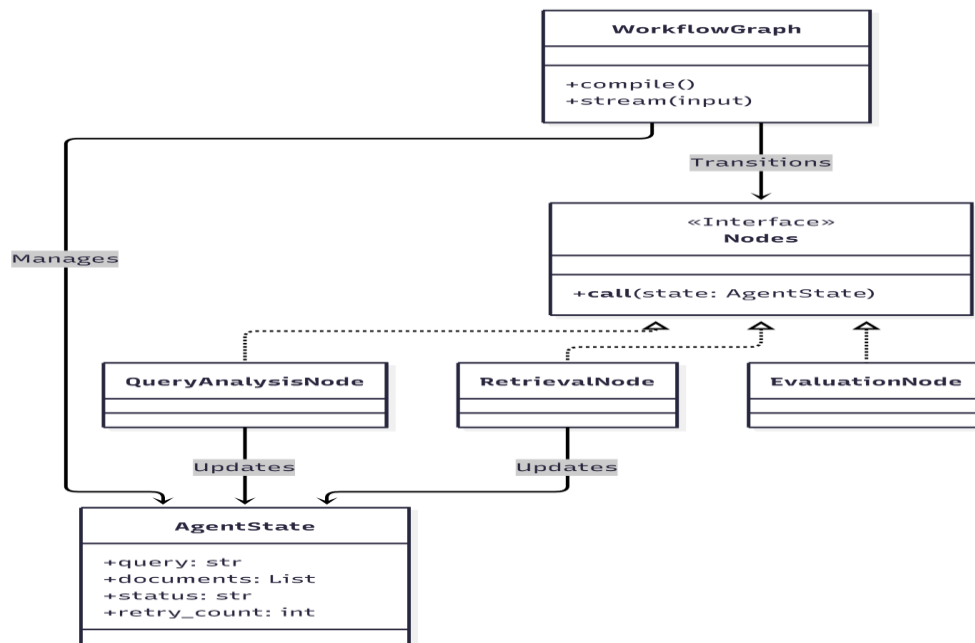


Figure 19 state pattern

Data Access Object (DAO) Pattern

Description: The DAO pattern provides an abstract interface to some type of database or other persistence mechanism. By mapping application calls to the persistence layer, the DAO provides some specific data operations without exposing details of the database.

The use of the DAO pattern improves the system:

- **Abstraction of Complexity:** The Agent calls `get_hadith_by_id(1)`. The DAO handles the ugly details: connecting to Chroma, querying the collection, fetching metadata, and handling missing IDs.
- **Domain-Persistence Separation:** If we were to decide to switch from ChromaDB to PostgreSQL later, you only change the `HadithRepository`. The `RetrievalAgent` (Business Logic) remains untouched because it relies on the `Document` object structure, not the database schema.

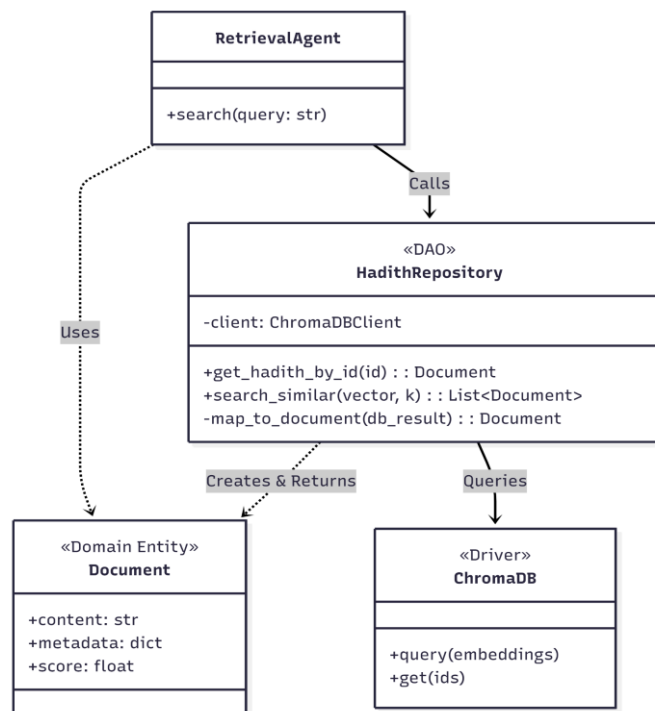


Figure 20 DAO pattern

Chapter 6 Implementation

1. Introduction

In this section, we will present the practical implementation of the Agentic Retrieval System, detailing the technologies and tools employed. Additionally, we will showcase the system interfaces and conclude by executing test cases to verify the system's functionalities across different scenarios.

2. Used tools and techniques

1. **FastAPI:**

FastAPI is used as the high-performance backend framework for building and serving the system's core APIs. It leverages Python's asynchronous capabilities to handle concurrent requests efficiently, making it ideal for the low-latency requirements of a RAG pipeline. It serves as the primary interface for the LangGraph orchestrator, facilitating the stateful execution of agents.

2. **LangGraph:**

LangGraph is utilized as the core orchestration engine for managing complex, stateful multi-agent workflows. Unlike linear chains, it enables cyclic execution patterns, allowing the system to implement self-correction loops between the Retrieval and Evaluation agents. It maintains the global AgentState across distinct execution nodes, ensuring contextual consistency and providing a robust framework for autonomous decision-making within the RAG pipeline.

3. **Streamlit:**

Streamlit is used to develop the interactive user interface for the system. It facilitates the rapid creation of a responsive frontend that connects directly to the FastAPI backend. Streamlit provides a seamless chat interface for users to submit queries and visualize retrieved evidence, while supporting

real-time updates and sidebars for system configuration and metadata filtering.

4. **ChromaDB:**

ChromaDB serves as the specialized vector database for high-dimensional document storage and retrieval. It is used to index Hadith chunks alongside their 1024-dimensional semantic embeddings generated by the BGE-M3 model. The system leverages ChromaDB's hybrid querying capabilities to combine dense vector similarity search with structured metadata filtering (e.g., collection, narrator, or chapter IDs), ensuring high precision in evidence retrieval.

5. **Visual studio code (VS code):**

Visual Studio Code combines the simplicity of a source code editor with powerful developer tooling, like IntelliSense code completion and debugging. We use it to develop the whole project (frontend, backend).

6. **GitHub:**

GitHub is a Git-based source code hosting platform that enables version control and collaborative software development. It provides tools for code management, issue tracking, and continuous integration, streamlining the development lifecycle and ensuring efficient team collaboration repository link.

7. **Diagrams:**

Web-based applications are used for modeling requirements and creating necessary charts, ensuring clear documentation and visualization of system components.

3. Test cases execution

TC ID	Test Case Title	Tested Data / Input	Expected Result	Actual Result	Pass/Fail
TC-QA-A-04	Verify Query Analysis Orchestration	Query: "Wudu steps and prayer times"	Agent outputs a structured "Search Plan" with split queries.	Plan generated with 2 sub-queries: "Wudu steps", "Prayer times".	PASS
TC-QA-A-05	Verify Typo Correction Tool	Query: "الصلات" (Spelling error)	System corrects query to "الصلاة".	Corrected query: "الصلاة" recorded in state.	PASS
TC-QA-A-06	Verify Query Normalization Tool	Query: "كِتَابُ الْإِيمَانِ" (Diacritics)	Text normalized to "كتاب الايمان".	Text normalized, diacritics removed successfully.	PASS
TC-QA-A-07	Verify Intent Classification Tool	Query: "Hadiths narrated by Abu Hurairah"	Intent classified as narrator_search.	Intent identified as narrator_search.	PASS
TC-QA-A-08	Verify Query Decomposition Tool	Query: "What is fasting and what breaks it?"	Split into: 1. "Definition of fasting", 2. "Invalidators of fasting".	Input decomposed into 2 distinct atomic queries.	PASS

TC-QA-A-09	Verify Collection Target Detection	Query: "Hadith about intention in Bukhari"	Target collection set to sahih_bukhari.	target_collection metadata set to ['bukhari'].	PASS
TC-QA-A-10	Verify Input Source Identification	Input: User pasted paragraph + "Explain this"	Source identified as user_provided_text.	Source classified as user_provided_text.	PASS
TC-RA-11	Verify Retrieval Planning Execution	Plan: [Sub-query 1, Sub-query 2]	Agent executes searches for both sub-queries.	Retrieval loop executed twice; results combined.	PASS
TC-RA-12	Verify Semantic Search Tool	Query: "Cleanliness ritual" (No exact keywords)	Retrieves Hadiths containing "Wudu" or "Ghusl".	Retrieved Hadith #135 about Wudu (Semantic match).	PASS
TC-RA-13	Verify Keyword Search Tool	Query: "Yarmouk"	Retrieves docs containing exact term "Yarmouk".	Retrieved 3 docs containing "Yarmouk".	PASS
TC-RA-14	Verify Metadata Filter Tool	Filter: narrator="Aisha"	All results have narrator: Aisha.	10/10 results verified as narrated by Aisha.	PASS

TC-RA-15	Verify Hybrid Search Tool	Query: "Zakat on camels"	Combines Vector and Keyword results.	Results list contains matches from both search types.	PASS
TC-RA-16	Verify Result Aggregation Tool	Input: 2 lists of 50 docs (overlapping)	Deduplicated, sorted list of unique docs.	Final list reduced to 65 unique docs, sorted by score.	PASS
TC-RA-17	Verify User-Provided Hadith Processor	Input: Custom text "X"	Search restricted to text "X".	Answer generated solely from provided text context.	PASS
TC-RA-18	Verify Document Ingestion (Runtime)	Upload: test_doc.pdf	Content from PDF is retrievable.	Query successfully retrieved sentence from test_doc.pdf.	PASS
TC-RA-19	Verify Document Reassembly	Query: "Hadith of Ka'b bin Malik" (Chunked)	Full text (Chunks 1-5) displayed as one.	All 5 chunks concatenated; full narrative displayed.	PASS
TC-RA-20	Verify Find Chapter Tool	Query: "Inheritance"	Returns Chapter ID for "Kitab al-Fara'id".	Returned Chapter ID 85 (Al-Fara'id).	PASS

TC-EA-21	Verify Evaluation Agent Logic	Input: Query + Retrieved Docs	Output structured Eval Object (Status/Reasoning).	Returned JSON: {"status": "PASS", "score": 0.8}.	PASS
TC-EA-22	Verify Quality Assessment Tool	Input: Mismatched Query/Docs	Quality score < 0.5 (Low).	Score: 0.25. Marked as irrelevant.	PASS
TC-EA-23	Verify Gap Identification Tool	Query: "5 Pillars", Docs: Mention only 3	Lists missing pillars as "Gaps".	Output: "Missing info: Zakat and Hajj details".	PASS
TC-EA-24	Verify Confidence Scoring Tool	Input: High relevance docs	Confidence score > 0.85.	Calculated Confidence: 0.92.	PASS
TC-EA-25	Verify Stopping Condition Tool	Input: Score 0.9 (Threshold 0.7)	Decision: STOP (Generate Answer).	Workflow routed to End/Synthesis.	PASS
TC-EA-26	Verify Grounding Validation Tool	Input: Answer with unsupported claim	Unsupported claim removed/flagged.	Claim about specific date removed (not in source).	PASS
TC-DIC-33	Verify Data Ingestion Pipeline	File: raw_bukhari.json	Pipeline runs, data searchable in DB.	Ingestion complete. Data	PASS

				verified in Vector Store.	
TC-DIC-34	Verify JSON Parsing	Field: hadith_no	extracted as integer.	hadith_no parsed correctly for 100% of sample.	PASS
TC-DIC-35	Verify Text Preprocessing	Input: Text with HTML tags	Output: Clean plain text.	HTML tags removed, text is clean.	PASS
TC-DIC-36	Verify Embedding Generation	Input: "Prophet said..."	Returns valid float vector.	Vector array [0.034, -0.012...] generated.	PASS
TC-DIC-37	Verify Vector Storage	Action: Query ID 100 in Chroma	Record exists with metadata.	Record found. Metadata matches source JSON.	PASS
TC-FI-38	Verify External AI Service Integration	Action: Call Modal Endpoint	Response 200 OK.	API returned valid response in 1.2s.	PASS
TC-FI-39	Verify LangGraph Workflow Engine	Action: Run standard query	Flow: Start → Analysis → Retrieval → Eval.	Graph edges traversed correctly as defined.	PASS

TC-FI-40	Verify Agent State Management	Action: Multi-step retry	State preserves context (Query & Docs).	AgentState retained history across 2 iterations.	PASS
TC-FI-41	Verify LangSmith Observability	Action: Check Dashboard	Trace visible for session.	Full trace log confirmed in LangSmith UI.	PASS
TC-INF-42	Verify Generative Reasoning Engine	Prompt: "Summarize findings"	Coherent summary generated.	Summary logically sound and grammatically correct.	PASS
TC-INF-43	Verify Vector Encoding Service	Batch: 10 texts	Returns 10 vectors.	10 vectors returned with correct dimensions (1024).	PASS
TC-INF-44	Verify Contextual Relevance Optimizer	Input: Mixed list	Relevant docs moved to top.	Top 5 docs are highly relevant (Cross-Encoder verified).	PASS
TC-UI-48	Verify Readability of Final Answer	Output: Arabic Text	RTL alignment, clear font.	Text displayed in RTL format, readable on UI.	PASS

Table 14 Test cases execution

4. Requirements Traceability Matrix (RTM)

Req_ID	Requirement	analysis	Detailed design	coding	Test cases
FR-SA-01	Supervisor Agent	analysis	design	source	TC-FI-39
FR-SA-02	Agent Task Assignment Tool	analysis	design	source	TC-FI-40
FR-SA-03	Iteration Control Tool	analysis	design	source	TC-FI-40
FR-QAA-04	Query Analysis Agent	analysis	design	source	TC-QAA-04
FR-QAA-05	Typo Correction Tool	analysis	design	source	TC-QAA-05
FR-QAA-06	Query Normalization Tool	analysis	design	source	TC-QAA-06
FR-QAA-07	Intent Classification Tool	analysis	design	source	TC-QAA-07
FR-QAA-08	Query Decomposition Tool	analysis	design	source	TC-QAA-08
FR-QAA-09	Collection Target Detection Tool	analysis	design	source	TC-QAA-09
FRQ-AA-10	Input Source Identification Tool	analysis	design	source	TC-QAA-10
FR-RA-11	Retrieval Planning Agent	analysis	design	source	TC-RA-11
FR-RA-12	Semantic Search Tool	analysis	design	source	TC-RA-12
FR-RA-13	Keyword Search Tool	analysis	design	source	TC-RA-13
FR-RA-14	Metadata Filter Tool	analysis	design	source	TC-RA-14
FR-RA-15	Hybrid Search Tool	analysis	design	source	TC-RA-15
FR-RA-16	Result Aggregation Tool	analysis	design	source	TC-RA-16
FR-RA-17	User-Provided Hadith Processor Tool	analysis	design	source	TC-RA-17
FR-RA-18	Document Ingestion Tool	analysis	design	source	TC-RA-18
FR-RA-19	Document Reassembly	analysis	design	source	TC-RA-19

FR-RA-20	Find chapter tool	analysis	design	source	TC-RA-20
FR-EA-21	Evaluation Agent	analysis	design	source	TC-EA-21
FR-EA-22	Quality Assessment Tool	analysis	design	source	TC-EA-22
FR-EA-23	Gap Identification Tool	analysis	design	source	TC-EA-23
FR-EA-24	Confidence Scoring Tool	analysis	design	source	TC-EA-24
FR-EA-25	Stopping Condition Tool	analysis	design	source	TC-EA-25
FR-EA-26	Grounding Validation Tool	analysis	design	source	TC-EA-26
FR-DIC-33	Data Ingestion Pipeline	analysis	design	source	TC-DIC-33
FR-DIC-34	JSON Parsing	analysis	design	source	TC-DIC-34
FR-DIC-35	Text Preprocessing	analysis	design	source	TC-DIC-35
FR-DIC-36	Embedding Generation	analysis	design	source	TC-DIC-36
FR-DIC-37	Vector Storage	analysis	design	source	TC-DIC-37
FR-FI-38	External AI Service Integration	analysis	design	source	TC-FI-38
FR-FI-39	LangGraph Workflow Engine	analysis	design	source	TC-FI-39
FR-FI-40	Agent State Management	analysis	design	source	TC-FI-40
FR-FI-41	LangSmith Observability Integration	analysis	design	source	TC-FI-41
FR-INF-42	Generative Reasoning Engine	analysis	design	source	TC-INF-42
FR-INF-43	Vector Encoding Service	analysis	design	source	TC-INF-43
FR-INF-44	Contextual Relevance Optimizer	analysis	design	source	TC-INF-44
FR-UI-45	Query History View	analysis	design	source	
FR-UI-46	Reasoning Trace Display	analysis	design	source	TC-FI-41
FR-UI-47	Relevance Feedback Tool	analysis	design	source	

FR-UI-48	Read Answer	analysis	design	source	TC-UI-48
FR-ADM-49	Knowledge Base Management Dashboard	analysis	design	source	
FR-ADM-50	Dynamic Configuration Interface	analysis	design	source	
FR-ADM-51	System Health Monitoring View	analysis	design	source	TC-FI-41

Table 15 RTM

Chapter 7 Conclusion

7.1 Conclusion

This project successfully developed an **Agentic Retrieval System** for Ṣaḥīḥ al-Bukhārī and Ṣaḥīḥ Muslim, bridging the technical gap between traditional Islamic scholarship and modern Artificial Intelligence. Unlike conventional search engines that rely on simple keyword matching, this system employs an autonomous **multi-agent architecture** orchestrated by LangGraph. By utilizing five specialized agents—Supervisor, Query Analysis, Retrieval, Evaluation, and Synthesis—the system performs iterative reasoning, allowing it to self-correct and refine search strategies dynamically until accurate information is found.

The technical implementation leveraged **Hybrid Search**, combining BGE-M3 semantic embeddings with BM25 lexical matching to ensure high precision for both thematic concepts and specific narrator names. Deploying the model on **Modal's serverless GPU infrastructure** ensured computational efficiency, while a strict **Zero-Hallucination policy** guaranteed that all generated answers were rigorously grounded in authenticated texts.

7.2 Future Work

Future development will focus on expanding the knowledge base to include the remaining books of the **Kutub al-Sittah** (Sunan Abu Dawood, Tirmidhi, etc.) and implementing **multilingual support** to allow global access to these resources. Additionally, integrating authentication chains (Isnad) and scholarly commentaries will further enhance the system's utility for legal and theological research. Ultimately, this project demonstrates that Agentic AI can serve as a reliable, rigorous assistant for navigating the profound depth of Prophetic traditions.

Appendices

- [Backend Repository](#)

References

Primary Data Sources

1. **Sahih al-Bukhari Dataset:**

- *Source:* The Hadith Collection Project (JSON format).
- *URL:* <https://github.com/AhmedBaset/hadith-json>
- *Description:* A structured collection of 7,277 authenticated narrations categorized by book and chapter.

2. **Sahih Muslim Dataset:**

- *Source:* The Hadith Collection Project (JSON format).
- *URL:* <https://github.com/AhmedBaset/hadith-json>
- *Description:* A structured collection of approximately 7,459 authenticated narrations.

Academic Papers & Articles

3. **Lewis, P., et al. (2020).** "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems (NeurIPS)*.

- *Context:* Foundational paper for RAG architecture.

4. **Yao, S., et al. (2023).** "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*.

- *Context:* The basis for the Agentic reasoning pattern used in this project.

5. **Torki, M., et al. (2023).** "ABCD: A Large-Scale Author-Based Hadith Classification Dataset."

- *Context:* Referenced in Literature Review regarding Deep Learning in Hadith studies.

6. **Alnefaie, R., et al. (2023).** "A Survey on Arabic Question Answering Systems: Challenges and Opportunities." *IEEE Access*.

- *Context:* Discusses the scarcity of religious QA datasets.

7. **Sudani, S. (2020).** "Semantic Search and Ontology Construction for Arabic Texts." *Journal of King Saud University - Computer and Information Sciences*.

8. **Zhang, Y., et al. (2024).** "Siren's Song in the AI Era: The Risk of Hallucinations in Large Language Models." *TechRxiv*.

Technical Documentation & Frameworks

9. **LangGraph Documentation:**

- *URL:* <https://langchain-ai.github.io/langgraph/>
- *Usage:* Orchestration of stateful multi-agent workflows.

10. **ChromaDB:**

- *URL:* <https://www.trychroma.com/>
- *Usage:* Vector database for storing BGE-M3 embeddings.

11. **Modal:**

- *URL:* <https://modal.com/>
- *Usage:* Serverless GPU hosting for Qwen2.5-14B model inference.

12. **BAAI General Embedding (BGE-M3):**

- *Source:* HuggingFace Hub.
- *Usage:* Multi-lingual embedding model optimized for semantic retrieval.

Islamic References

13. **Sunnah.com:** Online Hadith Database. (Accessed: Oct 2025).
14. **Dorar Sunni Encyclopaedia (HadithEnc):** <https://hadooth.net/> (Accessed: Oct 2025).